MaxSAT Evaluation 2024 : Solver and Benchmark Descriptions

2024

Berg , J , Järvisalo , M , Martins , R , Niskanen , A & Paxian , T (eds) 2024 , MaxSAT Evaluation 2024 : Solver and Benchmark Descriptions . Department of Computer Science Series of Publications B , vol. B-2024-2 , Department of Computer Science, University of Helsinki , Helsinki .

http://hdl.handle.net/10138/584878

publishedVersion

MaxSAT Evaluation 2024

Solver and Benchmark Descriptions

Jeremias Berg, Matti Järvisalo, Ruben Martins, Andreas Niskanen, and Tobias Paxian *(editors)*

University of Helsinki
Department of Computer Science
Series of Publications B
Report B-2024-2

ISSN 1458-4786
Helsinki 2024

PREFACE

The MaxSAT Evaluations (`https://maxsat-evaluations.github.io`) constitute a series of events focusing on the evaluation of current state-of-the-art systems for solving optimization problems via the Boolean optimization paradigm of maximum satisfiability (MaxSAT). Organized yearly starting from 2006, the year 2024 brought on the 19th edition of the MaxSAT Evaluations, organized as a satellite event of the 27th International Conference on Theory and Applications of Satisfiability Testing (SAT 2024). Some of the central motivations for the MaxSAT Evaluation series are to provide further incentives for further improving the empirical performance of the current state of the art in MaxSAT solving, to promote MaxSAT as a serious alternative approach to solving NP-hard optimization problems from the real world, and to provide the community at large heterogenous benchmark sets for solver development and research purposes.

The 2024 evaluation consisted of a total of five tracks: two for exact solvers (one for solvers focusing on unweighted and one for solvers focusing on weighted MaxSAT instances), two for in-exact MaxSAT solvers (using two short per-instance time limits, 60 and 300 seconds, differentiating from the per-instance time limit of 1 hour imposed in the main exact tracks), and the incremental track.

Solvers were required to be made available in open source, and a 1-2 page solver description was expected to accompany each solver submission, to provide some details on the search techniques implemented in the solvers. The solvers descriptions together with descriptions of new benchmarks for 2024 are collected together in this compilation.

Finally, we would like to thank everyone who contributed to MaxSAT Evaluation 2024 by submitting their solvers or new benchmarks. We are also grateful for the computational resources provided by the StarExec initiative and the Finnish Computing Competence Infrastructure (FCCI) which enabled running the 2024 evaluation smoothly.

Jeremias Berg, Matti Järvisalo, Ruben Martins, Andreas Niskanen, & Tobias Paxian
MaxSAT Evaluation 2024 Organizers

Contents

Benchmark Descriptions

SOLVERS

EvalMaxSAT 2024

Florent Avellaneda
Université du Québec à Montréal
Montréal, Canada
Email: avellaneda.florent@uqam.ca

Centre de Recherche de l'IUGM
Montréal, Canada
Email: florent.avellaneda@gmail.com

EvalMaxSAT[1] is a MaxSAT solver written in modern C++ language mainly using the Standard Template Library (STL). The solver is built on top of the SAT solver CaDiCaL [1], but any other SAT solver can easily be used instead. EvalMaxSAT is based on the OLL algorithm [2] originally implemented in the MSCG MaxSAT solver [3], [4] and then reused in the RC2 solver [5]. For a general description of how EvalMaxSAT functions, please refer to [6].

The 2024 version of EvalMaxSAT includes a few minor changes:

- To obtain an initial upper bound, only the NuWLS solver [7] is used for 3 minutes.
- Searching for a solution with SCIP is limited to 400 seconds and only if the maximum weight for every soft clause is under 500 million.
- The formula is preprocessed with an adaptation of the SBVA tool [8] if it contains fewer than 100 million hard clauses, and this preprocessing is limited to a maximum of 3 minutes.
- During the search for an unsat core, found but discarded cores are saved to be reconsidered in subsequent iterations without the need to rediscover them.

[1]https://github.com/FlorentAvellaneda/EvalMaxSAT

References

[1] A. Biere, K. Fazekas, M. Fleury, and M. Heisinger, "CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020," in *Proc. of SAT Competition 2020 – Solver and Benchmark Descriptions*, ser. Department of Computer Science Report Series B, T. Balyo, N. Froleyks, M. Heule, M. Iser, M. Järvisalo, and M. Suda, Eds., vol. B-2020-1. University of Helsinki, 2020, pp. 51–53.

[2] A. Morgado, C. Dodaro, and J. Marques-Silva, "Core-guided MaxSAT with soft cardinality constraints," in *Principles and Practice of Constraint Programming - 20th International Conference, CP 2014, Lyon, France, September 8-12, 2014. Proceedings*, ser. Lecture Notes in Computer Science, B. O'Sullivan, Ed., vol. 8656. Springer, 2014, pp. 564–573. [Online]. Available: https://doi.org/10.1007/978-3-319-10428-7_41

[3] A. Morgado, A. Ignatiev, and J. Marques-Silva, "MSCG: robust core-guided maxsat solving," *J. Satisf. Boolean Model. Comput.*, vol. 9, no. 1, pp. 129–134, 2014. [Online]. Available: https://satassociation.org/jsat/index.php/jsat/article/view/127

[4] A. Ignatiev, A. Morgado, V. M. Manquinho, I. Lynce, and J. Marques-Silva, "Progression in maximum satisfiability," in *ECAI 2014 - 21st European Conference on Artificial Intelligence, 18-22 August 2014, Prague, Czech Republic - Including Prestigious Applications of Intelligent Systems (PAIS 2014)*, ser. Frontiers in Artificial Intelligence and Applications, T. Schaub, G. Friedrich, and B. O'Sullivan, Eds., vol. 263. IOS Press, 2014, pp. 453–458. [Online]. Available: https://doi.org/10.3233/978-1-61499-419-0-453

[5] A. Ignatiev, A. Morgado, and J. Marques-Silva, "RC2: an efficient maxsat solver," *J. Satisf. Boolean Model. Comput.*, vol. 11, no. 1, pp. 53–64, 2019. [Online]. Available: https://doi.org/10.3233/SAT190116

[6] F. Avellaneda, "A short description of the solver evalmaxsat," *MaxSAT Evaluation*, vol. 8, 2020.

[7] Y. Chu, S. Cai, and C. Luo, "Nuwls: Improving local search for (weighted) partial maxsat by new weighting techniques," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 4, 2023, pp. 3915–3923.

[8] A. Haberlandt, H. Green, and M. J. Heule, "Effective auxiliary variables via structured reencoding," *arXiv preprint arXiv:2307.01904*, 2023.

Loandra in the 2024 MaxSAT Evaluation

Jeremias Berg, Christoph Jabs, Hannes Ihalainen, Matti Järvisalo
Department of Computer Science, University of Helsinki, Finland

I. Introduction and Preliminaries

We briefly overview the any-time Loandra MaxSAT-solver as it participated in the incomplete track of the 2024 MaxSAT Evaluation, focusing especially on the differences between the 2023 and 2024 versions. The base algorithm implemented in Loandra continues to be core-boosted linear search as first proposed for MaxSAT [3] and later extended to PBO and CP [10], [9]. The main difference to the previous years' versions of Loandra is the introduction of a local search component based on NuWLS [6], [7] (based on code from https://github.com/shaowei-cai-group/NuWLS-c), and changing the encoding of the PB constraints to the Dynamic Polynomial Watchdog (DPW) [18]. Loandra uses the RustSAT 0.5.1 (https://github.com/chrjabs/rustsat/releases/tag/rustsat-v0.5.1) implementation of the DPW through its c-API.

We assume familiarity with conjunctive normal form (CNF) formulas and weighted partial maximum satisfiability (MaxSAT). Treating a CNF formula as a set of clauses a MaxSAT instance $\mathcal{F}$ consists of two CNF formulas, the hard clauses F_h and the soft clauses F_s, as well a weight w_c associated with each $C \in F_s$. A solution to $\mathcal{F}$ is an assignment τ that satisfies F_h. The cost $\text{COST}(\mathcal{F}, \tau)$ of a solution τ is the sum of weights of the soft clauses falsified by τ. A solution is optimal if it has minimum cost. The cost $\text{COST}(\mathcal{F})$ of an instance is the cost of its optimal solutions. An unsatisfiable core κ of $\mathcal{F}$ is a subset of soft clauses s.t. $F_h \wedge \kappa$ is unsatisfiable. Loandra is implemented on top of Open-WBO [15]. We thank the developers of Open-WBO and NuWLS for their work.

II. Structure of Loandra

Figure 1 overviews the structure of Loandra. The solver implements core-boosted linear search [3] augmented with tightly integrated MaxSAT preprocessing [12], [2], [13], [14] and local search [6], [7]. More specifically, Loandra consists of four main components: a) *preprocessing*, b) *core-guided search* c) *solution improving search (SIS)*, and d) *local search*.

a) Preprocessing: On input $\mathcal{F}$, the execution starts by invoking the MaxPre 2.0 [13] preprocessor on $\mathcal{F}$. MaxPre 2.0 is run with the technique string `[u]#[uvsrgVGc]`, enforcing a 30s time-limit on and a skip technique value of 20. In more detail, the preprocessor runs the same "base" techniques as in previous years (unit propagation, bounded variable elimination, subsumption elimination, self-subsuming resolution, group subsumed label elimination, and binary core removal) as well as the so-called intrinsic at-most-one and TrimMaxSAT techniques [11], [17]. The TrimMaxSAT technique is extended to all literals rather than only literals appearing in soft clauses.

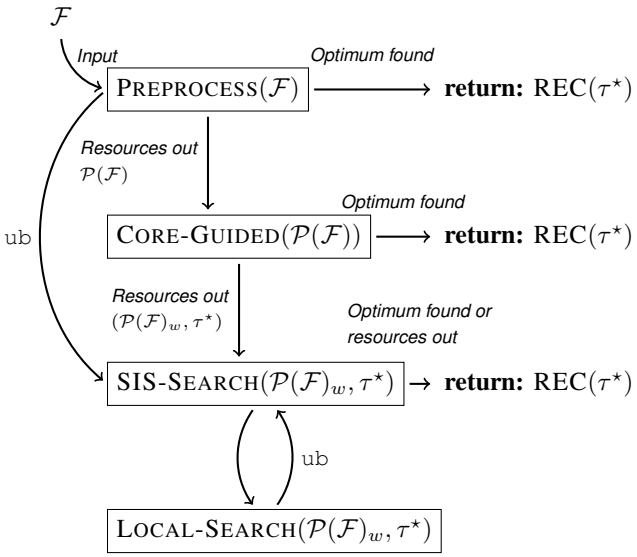


Fig. 1: The structure of Loandra.

In addition to simplifying the instance, MaxPRE 2.0 can obtain an upper bound `ub` on $\text{COST}(\mathcal{F})$. The bound is supplied to the linear search phase. Unless MaxPre can compute an optimal solution to $\mathcal{F}$, the core-guided search phase is then invoked on the preprocessed instance $\mathcal{P}(\mathcal{F})$. The core-guided phase reuses the assumption variables introduced during preprocessing [2].

b) Core-Guided Search: The core-guided phase is unchanged from previous versions of Loandra. As the instantiation of the core-guided algorithm, we use a reimplementation of PMRES [16] extended with weight aware core extraction (WCE) [4], stratification, and clause hardening. If CORE-GUIDED is able to find an optimal solution τ to the preprocessed instance $\mathcal{P}(\mathcal{F})$, an optimal solution $\text{REC}(\tau)$ to the original instance $\mathcal{F}$ is reconstructed and returned. Otherwise, the linear search component is invoked on the final working instance $\mathcal{P}(\mathcal{F})_w$ that is also initialized with the best-found solution τ^* during the core-guided phase.

c) Solution Improving Search: The SIS phase of Loandra is an implementation of the SAT/UNSAT linear search algorithm [5] extended with solution-guided phase saving and varying resolution in the style of LinSBPS [8]. The pseudo-boolean constraint is encoded with the so-called dynamic polynomial watchdog encoding [18], Loandra makes use of

the implementation of the DPW from RustSAT via its c-API. The initial bound $B = \min\{\mathrm{ub}, \mathrm{COST}(\mathcal{F}, \tau^*)\}$ on the PB-encoding is the minimum of the upper bound found by the preprocessor and the cost of the best solution found by the core-guided phase. As detailed in the next section, the SIS phase periodically invokes a local search solver: once at the beginning of SIS and once at the beginning of each new resolution.

SIS runs until either finding an optimal solution or running out of time, at which point a reconstruction $\mathrm{REC}(\tau^*)$ of the currently best-known solution τ^* to the preprocessed instance $\mathcal{P}(\mathcal{F})_w$ is returned. Notice that the reconstruction of a solution happens only once; we use the standard, linear time reconstruction algorithm as implemented in MaxPre.

d) Local Search: The local search component of Loandra is invoked at the beginning of the SIS phase and at the beginning of each new resolution (i.e. reinitialization of the SIS search with a larger set of soft clauses). The goal of the local search is twofold. On the one hand, we are looking for a good upper bound in order to decrease the size of the PB constraint encoded by SIS. On the other hand, as detailed in [14], we know that preprocessing in the context of anytime solving can result in misinterpretation of costs and the computation solutions whose costs are fairly easy to improve by local changes to the variable assignments.

III. Implementation details

All algorithms are implemented on top of the publicly available Open-WBO system [15] using Glucose 4.1 [1] as the back-end SAT solver. In order to minimize I/O overhead, we make direct use of the preprocessor interface offered by MaxPre. In the evaluation, we set a 30s time limit for the preprocessing phase and a 30s time limit for the core-guided phase. These limits were chosen based on preliminary experiments. The core-guided phase is also terminated on weighted instances when the stratification bound is lowered to 1. On unweighted instances, the phase is terminated at the latest after extracting one set of disjoint cores. As the local search component, we use NuWLS-c from https://github.com/shaowei-cai-group/NuWLS-c with its default settings.

IV. Compilation and Usage

Building and using Loandra resembles building and using Open-WBO. A statically linked version of Loandra in release mode can be built by running `MAKE RS` in the base folder. Note that you need to have an installation of Rust and cargo as well. Loandra is tested with Rust version 1.73.0

After building, Loandra can be invoked from the terminal. Except for the formula file, Loandra accepts several command line arguments: the flag *-pmreslin-cglim* sets the maximum time that the core-guided phase can run for (in seconds). The rest of the flags resemble the flags accepted by Open-WBO and MaxPRE; invoke ./loandra_static –help-verb for more information.

V. Acknowledgments

The algorithmic techniques underlying Loandra have been developed in collaboration with several different people. The solver's initial work was done with Emir Demirović and Peter Stuckey. Other contributors to Loandra include Marcus Leivo and Tuukka Korhonen. The Academy of Finland supports the primary developer under grant 1362987. My sincerest thanks to everyone who has contributed to the algorithmic ideas underlying Loandra.

References

[1] G. Audemard and L. Simon, "Predicting learnt clauses quality in modern sat solvers," in *Proc IJCAI*. Morgan Kaufmann Publishers Inc., 2009, pp. 399–404.

[2] J. Berg, P. Saikko, and M. Järvisalo, "Improving the effectiveness of SAT-based preprocessing for MaxSAT," in *Proc. IJCAI*. AAAI Press, 2015, pp. 239–245.

[3] J. Berg, E. Demirovic, and P. J. Stuckey, "Core-boosted linear search for incomplete maxsat," in *CPAIOR*, ser. Lecture Notes in Computer Science, vol. 11494. Springer, 2019, pp. 39–56.

[4] J. Berg and M. Järvisalo, "Weight-aware core extraction in SAT-based MaxSAT solving," in *Proc. CP*, ser. Lecture Notes in Computer Science, 2017, to appear.

[5] D. L. Berre and A. Parrain, "The sat4j library, release 2.2," *J. Satisf. Boolean Model. Comput.*, vol. 7, no. 2-3, pp. 59–6, 2010. [Online]. Available: https://satassociation.org/jsat/index.php/jsat/article/view/82

[6] S. Cai and Z. Lei, "Old techniques in new ways: Clause weighting, unit propagation and hybridization for maximum satisfiability," *Artif. Intell.*, vol. 287, p. 103354, 2020. [Online]. Available: https://doi.org/10.1016/j.artint.2020.103354

[7] Y. Chu, S. Cai, and C. Luo, "Nuwls: Improving local search for (weighted) partial maxsat by new weighting techniques," in *Proc AAAI*. AAAI Press, 2023, pp. 3915–3923.

[8] E. Demirovic and P. J. Stuckey, "Techniques inspired by local search for incomplete maxsat and the linear algorithm: Varying resolution and solution-guided search," in *Proc CP*, ser. LNCS, vol. 11802. Springer, 2019, pp. 177–194.

[9] J. Devriendt, S. Gocht, E. Demirovic, J. Nordström, and P. J. Stuckey, "Cutting to the core of pseudo-boolean optimization: Combining core-guided search with cutting planes reasoning," in *Proc AAAI*. AAAI Press, 2021, pp. 3750–3758.

[10] G. Gange, J. Berg, E. Demirovic, and P. J. Stuckey, "Core-guided and core-boosted search for CP," in *Proc CPAIOR*, ser. LNCS, vol. 12296. Springer, 2020, pp. 205–221.

[11] A. Ignatiev, A. Morgado, and J. Marques-Silva, "RC2: an efficient maxsat solver," *J. Satisf. Boolean Model. Comput.*, vol. 11, no. 1, pp. 53–64, 2019.

[12] H. Ihalainen, J. Berg, and M. Järvisalo, "Clause redundancy and preprocessing in maximum satisfiability," in *Proc IJCAR*, ser. LNCS, vol. 13385. Springer, 2022, pp. 75–94. [Online]. Available: https://doi.org/10.1007/978-3-031-10769-6_6

[13] T. Korhonen, J. Berg, P. Saikko, and M. Järvisalo, "Maxpre: An extended maxsat preprocessor," in *SAT*, ser. Lecture Notes in Computer Science, vol. 10491. Springer, 2017, pp. 449–456.

[14] M. Leivo, J. Berg, and M. Järvisalo, "Preprocessing in incomplete maxsat solving," in *ECAI*, ser. Frontiers in Artificial Intelligence and Applications, vol. 325. IOS Press, 2020, pp. 347–354.

[15] R. Martins, V. Manquinho, and I. Lynce, "Open-WBO: A modular MaxSAT solver," in *Proc. SAT*, ser. Lecture Notes in Computer Science, vol. 8561. Springer, 2014, pp. 438–445.

[16] N. Narodytska and F. Bacchus, "Maximum satisfiability using core-guided MaxSAT resolution," in *Proc. AAAI*. AAAI Press, 2014, pp. 2717–2723.

[17] T. Paxian, P. Raiola, and B. Becker, "On preprocessing for weighted maxsat," in *VMCAI*, ser. Lecture Notes in Computer Science, vol. 12597. Springer, 2021, pp. 556–577.

[18] T. Paxian, S. Reimer, and B. Becker, "Dynamic polynomial watchdog encoding for solving weighted maxsat," in *Proc SAT*, ser. LNCS, vol. 10929. Springer, 2018, pp. 37–53.

MaxSAT Evaluation 2024: Solver and Benchmark Descriptions, volume B-2024-2 of Department of Computer Science Series of Publications B, University of Helsinki 2024.

Exact: evaluating a pseudo-Boolean solver on MaxSAT problems (2024)

Jo Devriendt
Nonfiction Software
Koksijde, Belgium
jo.devriendt@nonfictionsoftware.com

***Abstract*—Weighted MaxSAT solving is a special case of pseudo-Boolean optimization, also known as binary linear programming. This submission aims to evaluate Exact, a conflict-driven cutting planes learning pseudo-Boolean solver, on MaxSAT problems.**

***Index Terms*—binary linear programming, pseudo-Boolean solving, cutting planes, core-guided optimization**

I. Introduction

It is well-known[1] that a weighted MaxSAT formula can be written as a binary linear program (BLP):

$$\begin{aligned} \text{Minimize} \quad & \sum_{c\in C} w_c z_c \\ \text{s.t.} \quad & z_c + \sum_{x\in c^+} x + \sum_{y\in c^-} (1-y) \geqslant 1 \qquad \forall c \in C \end{aligned}$$

where C is a set of clauses, c^+ and c^- are the set of positive and negative literals in a clause $c \in C$ respectively, w_c is the cost of not satisfying c, and all variables x, y and z are binary.

Even though this BLP formulation is natural, the state-of-the-art in previous MaxSAT evaluations employs repeated calls to Boolean satisfiability (SAT) solvers instead of one straightforward call to an integer linear programming (ILP) solver. The reason for this, perhaps, is that ILP solvers rely heavily on exploiting the linear relaxation of a BLP. However, as all constraints in the above BLP are clauses, its linear relaxation is particularly weak, negating the main strength of an ILP solver.

A third technology that could natively handle the above BLP is pseudo-Boolean (PB) solving. Similar to ILP technology, PB technology natively reasons on the linear constraints over binary variables present in the input. However, in contrast to ILP solvers, a PB solver does not chiefly depend on reasoning on the linear relaxation of a BLP. Instead, so-called *conflict-driven cutting-planes learning* (CDCPL) PB solvers derive (*learn*) from each conflict in the search tree an implied linear constraint that, if it had been derived previously, would have prevented the conflict through unit propagation. In this way, CDCPL PB solvers are a generalization of *conflict-driven clause learning* (CDCL) SAT solvers, where a CDCPL solver can learn stronger constraints than clauses.

[1]See, e.g., https://en.wikipedia.org/wiki/Maximum_satisfiability_problem#(1-1/e)-approximation

II. Submission

We submit the CDCPL solver Exact[2] to the MaxSAT evaluation. Exact is a fork of the CDCPL solver RoundingSat[3] [1]. For this submission, we do not employ RoundingSat's linear programming integration [2], as we expect the linear relaxations of the instances to be too weak. We do make use of its optimized propagation routines [3] and its hybrid core-guided optimization technique [4].

Exact improves upon its predecessor through a myriad of refactorings, extensions and improvements. We highlight three important ones for this MaxSAT evaluation submission.

A first one is the *stratification* routine of Exact's core-guided optimization. Instead of core-guided stratification based on [5], Exact uses a simple routine that ignores all soft clauses with a cost lower than some τ, which initially is set to the highest clause cost (i.e., the highest weight in the objective of the BLP representation). If Exact does not find a core with this τ (i.e., it finds a solution where all hard and non-ignored soft clauses are satisfied, or reaches timeout in the core-guided search) τ is halved to consider more soft clauses. This process is repeated until the maximum cost is halved to 1, at which point all soft clauses are taken into account.

A second improvement is the exploitation of the observation that a single *PB core* may yield multiple *cardinality cores*, which can be used during the core-guided lower bound derivation and objective reformulation process [4]. For instance, given an objective function $4x+3y+2z+w$ to be minimized, and a PB core $2x + 2y + z + w \geqslant 4$, Exact constructs an initial implied cardinality core $x + y + z \geqslant 2$, reformulating the objective to $2x + y + w + 2a + 4$ through the *extension constraint* $x + y + z = 2 + a$. But as $2x + 2y + z + w \geqslant 4$ also implies $x + y + w \geqslant 2$, Exact can further reformulate the objective to $x + 2a + b + 6$ with the extension constraint $x + y + w = 2 + b$, increasing the objective lower bound from 4 to 6 without any new core-guided solver call.

A third improvement is meant to address the fact that, given a search conflict implied only by clausal constraints, CDCPL solvers also learn only a clause, which is identical to regular CDCL SAT solving (which allows a more efficient clause-only implementation). For CDCPL to work well, non-clausal constraints need to appear in the conflict implication graph,

[2]https://gitlab.com/JoD/exact
[3]https://gitlab.com/miao_research/roundingsat

so that strong non-clausal constraints can be learned [6]. On MaxSAT instances, Exact introduces non-clausal constraints in three ways. Firstly, a derived upper or lower bound on the objective function is typically a non-clausal constraint. Secondly, a core-guided extension constraint is non-clausal. Thirdly, implied cardinality constraints can be detected from a conjunction of clauses. Existing research on cardinality detection in RoundingSat processes the implication graph during conflict analysis to obtain the right information to construct cardinality constraints [7]. Exact uses a different approach, repeatedly *probing* (deciding a single variable and running unit propagation) to construct the necessary edges in the implication graph to derive *at-most-one* cardinality constraints.

In previous years, Exact happily served as last-place contender in the competition, suggesting that at least this implementation of CDCPL is not very competitive on MaxSAT problems. Since we did not make significant changes aimed at MaxSAT problems, we expect the gap with specialized solvers to widen. Still, participating in the competition is a way of keeping in touch with the MaxSAT community. For instance, being included in MaxSAT fuzzing studies [8] did improve Exact's codebase.

III. Conclusion

By combining the effectiveness of CDCLP and core-guided optimization, PB solving technology may eventually become competitive to SAT-based approaches on MaxSAT problems. Exact's submission to this year's MaxSAT evaluation will provide experimental data to evaluate progress on this front.

IV. Acknowledgment

Exact is a fork of RoundingSat, which in turn uses code from MiniSat [9]. The author of Exact is Jo Devriendt (Nonfiction Software, formerly KU Leuven). The authors of RoundingSat are Jan Elffers (formerly Lund University), Jo Devriendt, Stephan Gocht (Lund University) and Jakob Nordström (University of Copenhagen, Lund University). The authors of MiniSat are Niklas Eén (formerly University of California) and Niklas Sörensson (formerly Chalmers University of Technology).

References

[1] J. Elffers and J. Nordström, "Divide and conquer: Towards faster pseudo-Boolean solving," in *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI '18)*, Jul. 2018, pp. 1291–1299.

[2] J. Devriendt, A. Gleixner, and J. Nordström, "Learn to relax: Integrating 0-1 integer linear programming with pseudo-Boolean conflict-driven search," in *Proceedings of the 17th International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR '20)*, ser. Lecture Notes in Computer Science, vol. 12296. Springer, Sep. 2020, pp. xxiv–xxv.

[3] J. Devriendt, "Watched propagation of 0-1 integer linear constraints," in *Proceedings of the 26th International Conference on Principles and Practice of Constraint Programming (CP '20)*, ser. Lecture Notes in Computer Science, vol. 12333. Springer, Sep. 2020, pp. 160–176.

[4] J. Devriendt, S. Gocht, E. Demirović, J. Nordström, and P. Stuckey, "Cutting to the core of pseudo-Boolean optimization: Combining core-guided search with cutting planes reasoning," vol. 35, no. 5, May 2021, pp. 3750–3758. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/16492

[5] C. Ansótegui, M. L. Bonet, J. Gabàs, and J. Levy, "Improving SAT-based weighted MaxSAT solvers," in *Proceedings of the 18th International Conference on Principles and Practice of Constraint Programming (CP '12)*, ser. Lecture Notes in Computer Science, vol. 7514. Springer, Oct. 2012, pp. 86–101.

[6] S. R. Buss and J. Nordström, "Proof complexity and SAT solving," in *Handbook of Satisfiability*, 2nd ed., ser. Frontiers in Artificial Intelligence and Applications, A. Biere, M. J. H. Heule, H. van Maaren, and T. Walsh, Eds. IOS Press, Feb. 2021, vol. 336, ch. 7, pp. 233–350.

[7] J. Elffers and J. Nordström, "A cardinal improvement to pseudo-Boolean solving," in *Proceedings of the 34th AAAI Conference on Artificial Intelligence (AAAI '20)*, Feb. 2020, pp. 1495–1503.

[8] T. Paxian and A. Biere, "Uncovering and classifying bugs in maxsat solvers through fuzzing and delta debugging," in *POS@SAT*, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:266363362

[9] N. Eén and N. Sörensson, "An extensible SAT-solver," in *6th International Conference on Theory and Applications of Satisfiability Testing (SAT '03), Selected Revised Papers*, ser. Lecture Notes in Computer Science, vol. 2919. Springer, 2004, pp. 502–518.

CGSS2 and CGSS2-AbstCG in the 2024 MaxSAT Evaluation

Hannes Ihalainen, Jeremias Berg, Matti Järvisalo
HIIT, Department of Computer Science, University of Helsinki, Finland

I. Introduction

CGSS2 is a MaxSAT solver implementing the core-guided OLL algorithm [12], [1], extended with weight aware core extraction, structure sharing and selective addition of equivalences as described in [9] and [4]. In MSE2024, the solver also implements the AbstCG algorithm for weighted MaxSAT [10], [11]. The CGSS2-AbstCG solver uses an OLL-AbstCG-hybrid. The solver reimplements in C++ the techniques of an older CGSS solver [9]. The solver makes use of stratification, hardening and the so-called core exhaustion, core minimization and intrinsic atmost1 techniques described in [7].

If you use CGSS2 in your research, we kindly ask you to cite [8] and [9].

II. Preliminaries

We assume familiarity with conjunctive normal form (CNF) formulas and weighted partial maximum satisfiability (MaxSAT). Treating a CNF formula as a set of clauses, a MaxSAT instance $\mathcal{F}$ consists of two CNF formulas, the hard clauses F_h and the soft clauses F_s, as well a weight $w(C)$ associated with each $C \in F_s$. A solution to $\mathcal{F}$ is an assignment τ that satisfies F_h. The cost of a solution τ is the sum of weights of the soft clauses falsified by τ. An optimal solution is one with minimum cost over all solutions. An unsatisfiable core κ of $\mathcal{F}$ is a subset of soft clauses s.t. $F_h \wedge \kappa$ is unsatisfiable.

Without loss of generality we assume that each soft clause is unit, containing the negation of a variable. We say that a variable b is an *objective variable* (of the instance $\mathcal{F}$) if $(\neg b) \in F_s$. As assigning a objective variable to 1 corresponds to falsifying a soft clause, we will in the rest of the text view cores as sets of objective variables and extend the weight function to objective variables via $w(b) = w((\neg b))$.

III. Main Features

We overview the main features of CGSS2. For a more detailed description, we refer the reader to [9] and [8].

OLL. When solving an instance $\mathcal{F}$ the (basic form of the) OLL algorithm [12], [1] iteratively extracts unsatisfiable cores of $\mathcal{F}$ using a SAT-solver, and then reformulates the instance in a way that allows exactly one of the objective variables in the core to be assigned to 1 (corresponding to falsifying a soft clause) in subsequent iterations. This continues until the SAT solver reports the reformulated instance to be satisfiable and returns an optimal solution of the original instance.

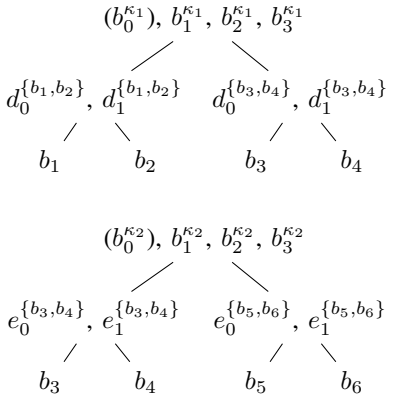


Fig. 1: The structure of totalizers built when relaxing cores $\kappa_1 = \{b_1, b_2, b_3, b_4\}$ (above) and $\kappa_2 = \{b_3, b_4, b_5, b_6\}$ (below).

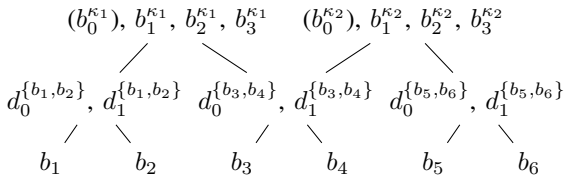


Fig. 2: The structure of totalizers when relaxing the cores κ_1 and κ_2 with structure sharing.

Core reformulation. For reformulating a core $\kappa = \{b_1, \ldots, b_n\}$, the CGSS2 solver uses the so called *totalizer* [3] CNF encoding of cardinality constraints. The totalizer encoding can be viewed as a tree structure similar to the ones depicted in Figure 1. The leafs of the tree correspond to the variables in the core. An internal node that is the root of a subtree with the set $S \subseteq \kappa$ as leaves corresponds to $|S| = m$ new variables $b_0^S, \ldots, b_{m-1}^S$ defined with clauses equivalent to $\left(\sum_{b \in S} b \geq k + 1\right) \rightarrow b_k^S$. Specifically the root of the full tree corresponds to a set $b_0^\kappa, \ldots b_{n-1}^\kappa$ that count the number of variables of κ set to true by assignments satisfying the totalizer.

Weight aware core extraction (WCE) [5]. WCE is a heuristic designed to delay the core-reformulation steps performed by a solver implementing OLL for as long as possible. When extracting a new core κ, a solver using WCE will lower the weight of each variable $b \in \kappa$ by $w^\kappa = \min\{w(b) \mid b \in \kappa\}$

(this correspond to the so called clause cloning step). Afterwards, the core is stored and the SAT-solver asked for another core containing variables with positive weight. The stored cores are only reformulated when no new cores can be found. Note that in the unweighted case (i.e. when the weight of each variable is 1) WCE is equivalent to the so called disjoint core technique that extracts a disjoint set of cores before reformulating.

Structure sharing. Structure sharing is a recently proposed refinement applied together with WCE that attempts to reduce the number of equivalent variables introduced by the core reformulation steps by identifying subtrees that can be shared between several different totalizers. For a concrete example, Figure 1 demonstrates two possible totalizer structures that can be built when relaxing the cores $\kappa_1 = \{b_1, b_2, b_3, b_4\}$ (above) and $\kappa_2 = \{b_3, b_4, b_5, b_6\}$ (below). Both of these structures include a subtree with b_3 and b_4 as leaves. The root of each of these subtrees define separate sets of variables ($\{d_0^{\{b_3,b_4\}}, d_1^{\{b_3,b_4\}}\}$ in the top tree, $\{e_0^{\{b_3,b_4\}} e_1^{\{b_3,b_4\}}\}$ in the bottom) that count the number of variables from the set $\{b_3, b_4\}$ set to true by assignments satisfying the totalizers. These variables will be assigned the same way by all satisfying assignments to the instance. Stated in another way, the two totalizer structures depicted in Figure 1 are equivalent to the smaller single structure depicted in Figure 2.

When relaxing a set of cores obtained via WCE, CGSS2 uses a heuristic set-covering algorithm for identifying maximal sets of literals shared by as many cores as possible and building totalizers that share these sets as subtrees.

Selective addition of equivalences. Consider a count variable b_k^S corresponding to an internal note of a tree that is the root of a subtree with the variables in S as leaves. For correctness of the OLL algorithm, it suffices to add clauses equivalent to the implication $\left(\sum_{b \in S} b \geq k+1\right) \rightarrow b_k^S$. While adding the other direction of the implication (i.e. $b_k^S \rightarrow \left(\sum_{b \in S} b \geq k+1\right)$) could allow the SAT solver to perform more propagation, the large number of clauses required in order to do so for every internal node might instead result in overall decrease in performance.

To balance the potential benefits and overhead (due to extra clauses) of adding both sides of the equivalence defining the variables in a totalizer, CGSS2 attempts to identify nodes for which the equivalence constraints are more likely to lead to further propagation. More specifically, for each leaf and root of a shared subtree, two values are computed: (a) the number of additional clauses needed for defining the full equivalence and (b) how many decisions need to performed by the SAT solver in before the additional constraints result in propagation. If both of these values are below some user provided threshold the equivalence constraints for that particular node are added.

SAT solver. As an underlying SAT-solver, CGSS2 supports Glucose 3 [2] and CaDiCaL [6]. In the evaluation setting, Glucose 3 is used. On each SAT solver call, CGSS2 sorts the assumptions by their weights in decreasing order.

Upper bounds. During search, CGSS2 may find intermediate solutions to the instance. Such solutions give upper bounds on the optimal cost. CGSS2 uses the upper bound for hardening soft clauses and for checking if the search can be terminated (if known lower and upper bounds for the optimal solution match). The bound-based termination may allow termination even without relaxing all found cores.

IV. Compilation and Usage

CGSS2 can be downloaded from https://bitbucket.org/coreo-group/cgss2/src/master/ or the Evaluation website. Instructions for compilation can be found in the README.md file.

In MSE 2024 the CGSS2 solver is invoked with the command line

```
./cgss2 [instance.wcnf]
```

CGSS2-AbstCG is invoked with the command line

```
./cgss2 --abst-cg [instance.wcnf]
```

References

[1] B. Andres, B. Kaufmann, O. Matheis, and T. Schaub, "Unsatisfiability-based optimization in clasp," in *Proc. ICLP Technical Communications*, ser. LIPIcs, vol. 17. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik, 2012, pp. 211–221.

[2] G. Audemard, J. Lagniez, and L. Simon, "Improving Glucose for incremental SAT solving with assumptions: Application to MUS extraction," in *Theory and Applications of Satisfiability Testing - SAT 2013 - 16th International Conference, Helsinki, Finland, July 8-12, 2013. Proceedings*, ser. Lecture Notes in Computer Science, M. Järvisalo and A. V. Gelder, Eds., vol. 7962. Springer, 2013, pp. 309–317. [Online]. Available: https://doi.org/10.1007/978-3-642-39071-5_23

[3] O. Bailleux and Y. Boufkhad, "Efficient CNF encoding of boolean cardinality constraints," in *Proc. CP*, ser. Lecture Notes in Computer Science, vol. 2833. Springer, 2003, pp. 108–122.

[4] J. Berg and M. Järvisalo, "Weight-aware core extraction in SAT-based MaxSAT solving," in *Proc. CP*, ser. Lecture Notes in Computer Science, 2017, to appear.

[5] J. Berg and M. Järvisalo, "Weight-aware core extraction in SAT-based MaxSAT solving," in *Proc. CP*, ser. Lecture Notes in Computer Science, vol. 10416. Springer, 2017, pp. 652–670.

[6] A. Biere, K. Fazekas, M. Fleury, and M. Heisinger, "CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020," in *Proc. of SAT Competition 2020 – Solver and Benchmark Descriptions*, ser. Department of Computer Science Report Series B, T. Balyo, N. Froleyks, M. Heule, M. Iser, M. Järvisalo, and M. Suda, Eds., vol. B-2020-1. University of Helsinki, 2020, pp. 51–53.

[7] A. Ignatiev, A. Morgado, and J. Marques-Silva, "RC2: An efficient MaxSAT solver," *Journal on Satisfiability, Boolean Modeling and Computation*, vol. 11, no. 1, pp. 53–64, 2019.

[8] H. Ihalainen, "Refined core relaxations for core-guided maximum satisfiability algorithms," MSc thesis, University of Helsinki, 2022, http://hdl.handle.net/10138/351207.

[9] H. Ihalainen, J. Berg, and M. Järvisalo, "Refined core relaxation for core-guided maxsat solving," in *CP*, ser. LIPIcs, vol. 210. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021, pp. 28:1–28:19.

[10] ——, "Unifying core-guided and implicit hitting set based optimization," in *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI 2023, 19th-25th August 2023, Macao, SAR, China*. ijcai.org, 2023, pp. 1935–1943. [Online]. Available: https://doi.org/10.24963/ijcai.2023/215

[11] ——, "Unifying sat-based approaches to maximum satisfiability solving," *Journal of Artificial Intelligence Research*, 2024.

[12] A. Morgado, C. Dodaro, and J. Marques-Silva, "Core-guided MaxSAT with soft cardinality constraints," in *Proc. CP*, ser. Lecture Notes in Computer Science, vol. 8656. Springer, 2014, pp. 564–573.

MaxCDCL in MaxSAT Evaluation 2024

1st Chu-Min Li
Université de Picardie Jules Verne,
MIS, Amiens, France
Aix Marseille Univ, *Université de Toulon*
CNRS, LIS, Marseille, France
chu-min.li@u-picardie.fr

2nd Shuolin Li*
Aix Marseille Univ, *Université de Toulon*
CNRS, LIS, Marseille, France
shuolin.li@etu.univ-amu.fr

3rd Jordi Coll
Universitat de Girona
Girona, Spain
jordicoll@udg.edu

4th Djamal Habet
Aix Marseille Univ, *Université de Toulon*
CNRS, LIS, Marseille, France
Djamal.Habet@univ-amu.fr

5th Felip Manyà
Artificial Intelligence Research Institute
CSIC, Bellaterra, Spain
felip@iiia.csic.es

6th Kun He
Huazhong University
of Science and Technology
Wuhan,China
brooklet60@hust.edu.cn

I. Introduction

MaxCDCL is an extension of CDCL (Conflict Driven Clause Learning) based on the Branch-and-Bound (BnB) scheme for unweighted (partial) MaxSAT [1]. Its main distinguishing feature w.r.t. a CDCL SAT solver is a lookahead procedure to underestimate a lower bound (LB) of the number of soft clauses that will be falsified if search continues. If LB reaches the current upper bound (UB), i.e., if LB$\geq$UB, then any solution better than the best solution found so far does not exist. This situation is called a soft conflict, from which an analysis is carried out to derive a hard clause explaining the soft conflict and to guide the backtracking, so that the same soft conflict will not be produced again in the future. When a hard clause is falsified, the conflict is said to be hard, and MaxCDCL analyzes it as in a CDCL SAT solver to learn a hard clause. The soft conflicts are together with the hard conflicts to drive the entire search, given UB.

MaxCDCL already participated in MaxSAT evaluations 2022 [2] and 2023 [3]. It was ranked 6th over 11 competitors in the unweighted partial MaxSAT track in 2022, and was ranked 2nd in 2023.

II. New techniques in MaxCDCL 2024

A. Reinforcement learning for lower bounding

MaxCDCL represents each soft clause using a soft literal. A local core w.r.t. a partial assignment α is a subset of soft literals that cannot be satisfied at the same time without falsifying a hard clause under α. Thus, an important property of a core w.r.t. α is that it contains at least a falsified soft literal if α is extended to a complete assignment. So, the number of disjoint local cores w.r.t. α is a lower bound (LB) of the number of falsified soft literals for any assignment extended from α.

Unfortunately, the detection of a local core is time-consuming, and if the computed LB does not reach UB, i.e., if the LB computation is not successful, the effort spent to compute LB is lost. Consequently, MaxCDCL does not detect local cores for all partial assignments. Instead, the previous versions of MaxCDCL use a probing technique to estimate the average a and standard deviation d of the number of disjoint local cores detected in a successful LB detection, and decides whether the LB computation should be hired for a partial assignment based on a and d. Concretely, let k be the number of disjoint local cores to detect for a successful LB computation, and c be a parameter. If $k \leq a + c * d$, then the LB computation should be hired. The intuition is as follows. Let n be the number of disjoint local cores to be detected in a successful LB computation. A reasonable hypothesis is that n is a random number with normal distribution. Then 95% (99%) of its values are between $a - c * d$ and $a + c * d$ when $c = 2$ (3) according to a well-known property in statistics. In practice, c is adjusted to maintain a successful rate between 60% and 75%.

In MaxCDCL2024, we use reinforcement learning with 2 armed-bandit mechanism to decide if LB computation should be hired. For this purpose, two actions, *yes* and *no*, are associated with each k, where k is the number of disjoint local cores for a LB computation to be successful. Initially, the score of *yes* and *no* is initialized to 0. If a LB computation is hired and is successful, then the score(*yes*) and score(*no*) are updated as follows.

$$score(yes) \leftarrow score(yes) + stepSize * (1 - score(yes))$$

$$score(no) \leftarrow score(no) * stepSize * (1 - score(no))$$

If the LB computation is not successful, then

$$score(yes) \leftarrow score(yes) * stepSize * (1 - score(yes))$$

$$score(no) \leftarrow score(no) + stepSize * (1 - score(no))$$

where $stepSize$ is a parameter initialized to 0.4 and is decreased by 10^{-5} before every LB computation until it reaches 0.04.

*Corresponding author

In summary, the conditions to hire LB computation in MaxCDCL2024 are as follows. For each new conflict, with a small probability (0.02), LB computation is always hired for exploration. Otherwise, LB computation should be hired if $c * score(yes) \geq score(no)$, where c is initialized to 1 and is adjusted to maintain the success rate of LB computation to be between 0.6 and 0.75.

B. Incremental lower bounding

In previous versions of MaxCDCL, the disjoint local cores are discarded immediately after a LB computation, whether the computation is successful or not. In MaxCDCL2024, if the LB computation is not successful, the detected cores are kept until backtracking or the next LB computation, which has two consequences:

- Let k be the number of disjoint local cores, and $k < UB$. If next branchings make n new falsified soft literals that do not belong to these cores, and $k + n \geq UB$, then MaxCDCL should backtrack, avoiding a new LB computation.
- Let $\{s_1, \ldots, s_d\}$ be a local core, if $d - 1$ literals, say, $s_2, \ldots, s_d$, are assigned true and s_1 is not assigned, then s_1 should be assigned false and propagated. Furthermore, let $\{h_1, \ldots, h_r\}$ be the reason of the core. The clause $\neg s_1 \vee h_1 \vee \cdots \vee h_r \vee \neg s_2 \vee \cdots \vee \neg s_d$ is created as the reason of $\neg s_1$.

C. Instances with few soft conflicts

MaxCDCL encounters very few soft conflicts when solving some particular instances. In other words, the time-consuming lookahead procedure is not effective for these instances and can learn clauses of low quality from the few soft conflicts. In MaxCDCL2024, the normal conditions to hire the lookahead procedure are executed for 200s. If $\#softConflicts/(\#softConflicts + \#hardConflicts) < 0.1$ after 200s, then the conditions for hiring lookahead become more restrictive for the rest of solving: The random call to lookahead at each new conflict is canceled, and at most one call to lookahead is hired after a new conflict even if the call is not successful, allowing to save time by avoiding the ineffective lookaheads.

III. Versions submitted to MSE2024

We submit two versions of MaxCDCL2024 to MSE2024. The first version is MaxCDCL2024 itself. The second version combines MaxCDCL2024 with Open-WBO [4] (version MSE2023) in a portfolio in which Open-WBO is first executed 300s, followed by MaxCDCL2024 for 3300s. Note that Open-WBO, WMaxCDCL2023 and MaxCDCL2023 are the only exact solvers of MSE2023 which pass successfully the regression test required in MSE2024.

Acknowledgments

This work is partially supported by AI CHAIR reference ANR-19-CHIA-0013-01 (MASSAL'IA) and project ANR-20-ASTR-0011 (POSTCRYPTUM) funded by the French Agence Nationale de la Recherche, and projects PID2019-111544GB-C21 and TED2021-129319B-I00 funded by MCIN/AEI/10.13039/501100011033, and partially supported by Archimedes Institute, Aix-Marseille University. We thank the Université de Picardie Jules Verne for providing the Matrics Platform.

References

[1] C.-M. Li, Z. Xu, J. Coll, F. Manyà, D. Habet, and K. He, "Combining clause learning and branch and bound for maxsat," in *27th International Conference on Principles and Practice of Constraint Programming (CP 2021)*, 2021.

[2] J. Coll, S. Li, C.-M. Li, F. Manya, D. Habet, and K. He, "Maxcdcl and wmaxcdcl in maxsat evaluation 2022," in *MaxSAT Evaluation 2022 : Solver and Benchmark Descriptions*. Department of Computer Science, University of Helsinki, 2022, pp. 15–16.

[3] C.-M. Li, J. Coll, S. Li, D. Habet, F. Manyà, and K. He, "Maxcdcl in maxsat evaluation 2023," *MaxSAT Evaluation 2023*, pp. 14–15, 2023.

[4] R. Martins, V. M. Manquinho, and I. Lynce, "Open-WBO: A modular MaxSAT solver," in *Proceedings of SAT 2014*, ser. LNCS, vol. 8561. Springer, 2014, pp. 438–445.

WMaxCDCL in MaxSAT Evaluation 2024

1st Shuolin Li
Aix Marseille Univ, *Université de Toulon*
CNRS, LIS, Marseille, France
shuolin.li@etu.univ-amu.fr

2nd Chu-Min Li*
Université de Picardie Jules Verne,
MIS, Amiens, France
Aix Marseille Univ, *Université de Toulon*
CNRS, LIS, Marseille, France
chu-min.li@u-picardie.fr

3rd Jordi Coll
Universitat de Girona
Girona, Spain
jordicoll@udg.edu

4th Djamal Habet
Aix Marseille Univ, *Université de Toulon*
CNRS, LIS, Marseille, France
Djamal.Habet@univ-amu.fr

5th Felip Manyà
Artificial Intelligence Research Institute
CSIC, Bellaterra, Spain
felip@iiia.csic.es

6th Kun He
Huazhong University of
Science and Technology
Wuhan, China
brooklet60@hust.edu.cn

***Abstract*—This document is an introduction of our submitted solvers to MaxSAT Evaluation 2024.**

I. Introduction

WMaxCDCL2024 solves Weighted Partial MaxSAT instances. It is developed from WMaxCDCL2023 [1] which won the weighted partial category of MaxSAT Evaluation 2023. Its main algorithm is MaxCDCL [3], an algorithm that combines **Branch and Bound(BnB)** and **clause learning**. As a BnB solver, it computes for a partial assignment a lower bound (LB) of the total weight of soft clauses that will be falsified when the partial assignment is extended, and compares LB with an imposed upper bound (UB). This process is called *lookahead*. If $LB \geq UB$, no solution better than UB can be found. So, the solver prunes and backtracks; otherwise the search continues. To better achieve clause learning, it transforms each soft clause c into a soft literal s with the same weight, so that the value of s is true if c is satisfied and vice versa. The soft clauses are removed from the formula after the transformation, and the solver works with soft literals.

Some new features have been added into WMaxCDCL2024 w.r.t. WMaxCDCL2023.

II. In-processing Bounded Variable Elimination and Equivalent Literal Substitution

The implementation details of Bounded Variable Elimination(BVE) and Equivalent Literal Substitution(ELS) are similar to those in MaxCDCL2023 [2], but some modifications are done in ELS to fit the property of Weighted Partial MaxSAT.

If soft literals s_1 and s_2 are complementary in MaxCDCL, i.e., s_1 is satisfied if and only if s_2 is falsified, then the constant cost is increased by 1, and both literals are removed, while in WMaxCDCL the literal with higher weight is kept and its weight updated to the difference of two weights. If soft literal s_1 and s_2 are equivalent, nothing is done in MaxCDCL, while in WMaxCDCL the two literals could be unified by removing one of them and the sum of their weights becomes the weight of the remaining one.

*Corresponding author

III. Improvements in Lookahead

In WMaxCDCL, the key to efficient pruning is to compute a high-quality LB for a partial assignment α. For this purpose, it detects local cores w.r.t. α using unit propagation, where a local core w.r.t. α is a subset of soft literals that cannot all be satisfied without falsifying a hard clause under α. So, the key property of a local core is that there is at least one soft literal falsified by any complete assignment extended from α, and we define the weight of a core to be the minimum weight of the soft literals it contains. After detecting the core, the weight of any soft literal in the core is reduced by the weight of the core. We say that a core locks a part of weight of each literal in it. To guarantee the soundness of LB, the locked weight cannot be considered in other cores, and only the soft literals with non-zero (remaining) weight participate in the detection of other cores. Thus, the sum of weights of all detected cores and of weights of all soft literal already falsified under α is a lower bound LB.

Two main improvements are introduced in lookahead in WMaxCDCL2024 that are described in this section.

A. Reinforcement learning for hiring Lookahead

Lookahead is time consuming, and if the computed LB does not reach UB, i.e., if the LB computation is not successful, the effort spent to compute LB is lost. Consequently, WMaxCDCL does not detect local cores for all partial assignments. Instead, the previous versions of WMaxCDCL use a probing technique to estimate the average a and standard deviation d of the number of disjoint local cores detected in a successful LB computation, and decides whether the LB computation should be hired for a partial assignment based on a and d.

In WMaxCDCL2024, we use reinforcement learning with 2 armed-bandit mechanism to decide if LB computation should be hired. Concretely, two actions, *yes* and *no*, are associated with each k, where k is the number of disjoint local cores for

a LB computation to be successful. Initially, the score of *yes* and *no* is initialized to 0. If a LB computation is hired and is successful, then the score are updated as follows.

$$score(yes) \leftarrow score(yes) + stepSize * (1 - score(yes))$$

$$score(no) \leftarrow score(no) * stepSize * (1 - score(no))$$

If the LB computation is not successful, then

$$score(yes) \leftarrow score(yes) * stepSize * (1 - score(yes))$$

$$score(no) \leftarrow score(no) + stepSize * (1 - score(no))$$

where $stepSize$ is a parameter initialized to 0.4 and is decreased by 10^{-5} before every LB computation until 0.04.

LB computation should be hired if $c * score(yes) \geq score(no)$, where c is initialized to 1 and is adjusted to maintain the success rate of LB computation to be between 0.6 and 0.75.

B. Unlocking Cores during Lookahead

In previous versions of WMaxCDCL, a soft literal with remaining weight 0 cannot participate in any further core, because all its weight is locked in existing cores. We say that such a literal is *locked*. The LB computation is based on the very conservative hypothesis that there is only one falsified soft literal s in each core and that s necessarily has the minimum weight in the core. In WMaxCDCL2024, we develop a core unlocking mechanism to detect cores that have more than one falsified soft literals, so that better LB can be computed.

We illustrate the core unlocking mechanism using an example. When we say we propagate a literal s, we mean that we assign true to s and execute unit propagation (UP); and when we say UP falsifies s, we mean that UP produced a hard unit clause $\neg s$. Let $c_1 = \{s_1, s_2, s_3\}$ be a core and the remaining weight of s_1 and s_2 is 0 so that they cannot participate in any further core in previous versions of WMaxCDCL. The left of Fig. 1 shows the detection of c_1: propagating s_1 and then s_2 falsifies s_3, meaning that s_1, s_2, s_3 cannot be all satisfied without falsifying a hard clause w.r.t. α.

To detect a new core, we propagate a soft literal s_4 with non-zero remaining weight. Suppose that UP falsifies s_1. Then we unlock c_1, so that s_2 can be propagated, which falsifies a soft literal s_5 with non-zero remaining weight. We now show that the subset $\{s_1, s_2, s_3, s_4, s_5\}$ has at least two falsified soft literals under any assignment extending α. If s_4 is false, then the two falsified literals are s_4 and one of s_1, s_2, s_3 because $\{s_1, s_2, s_3\}$ is a core (case 1). If s_4 is true, then the two falsified literals are s_1, s_2 (case 2), or s_1, s_5 (case 3) according to UP. The right of Fig. 1 shows the three cases modulo UP. Let $w(s)$ ($w(c)$) denote the weight of soft literal s (core c), where $w(s)$ is the sum of the remaining weight of s and the part of weight of s locked in c_1. The weight of the core $c_2 = \{s_1, s_2, s_3, s_4, s_5\}$ is defined to be the minimum among $w(s_4)+w(c_1)$, $w(s_2)+w(s_1)$ and $w(s_5)+w(s_1)$. The part of weight of s_1, s_2, s_3 locked in c_2 is equal to their part locked in c_1, and the part of weight of s_4 and s_5 locked in c_2 is equal to $w(c_2) - w(c_1)$. The remaining weight of all literals is computed accordingly. Finally, c_1 is removed.

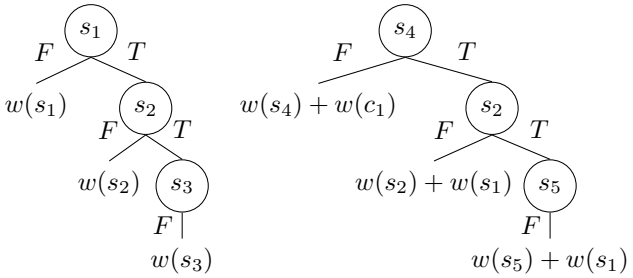


Fig. 1. Example illustrating core unlocking mechanism

Note that the new core $c_2 = \{s_1, s_2, s_3, s_4, s_5\}$ may not be detected if c_1 is not unlocked and s_2 is not propagated.

In the general case, an existing core is unlocked and its unassigned literals can be propagated during the detection of a new core once the total weight of falsified soft literals in the core is equal to or greater than the weight of the core.

In practice, WMaxCDCL2024 first executes normal lookahead without unlocking any existing core. If the computed LB does not reach UB because any new core cannot be detected after propagating all soft literals with non-zero weight, it re-executes lookahead with unlocking to detect more cores.

IV. Versions submitted to MSE2024

We submit two versions to MSE2024. The first is WMaxCDCL2024 itself. The second combines WMaxCDCL2024 and Open-WBO [4] (version MSE2023) in a portfolio in which Open-WBO is first executed 1200s, followed by WMaxCDCL2024. Note that Open-WBO, WMaxCDCL2023 and MaxCDCL2023 are the only exact solvers of MSE2023 which pass successfully the regression test required in MSE2024.

Acknowledgments

This work is partially supported by AI CHAIR reference ANR-19-CHIA-0013-01 (MASSAL'IA) and project ANR-20-ASTR-0011 (POSTCRYPTUM) funded by the French Agence Nationale de la Recherche, and projects PID2019-111544GB-C21 and TED2021-129319B-I00 funded by MCIN/AEI/10.13039/501100011033, and partially supported by Archimedes Institute, Aix-Marseille University. We thank the Université de Picardie Jules Verne for providing the Matrics Platform.

References

[1] Jordi Coll, Shuolin Li, Chu-Min Li, Felip Manyà, Djamal Habet, Mohamed Sami Cherif, and Kun He. Wmaxcdcl in maxsat evaluation 2023. *MaxSAT Evaluation 2023*, pages 16–17, 2023.

[2] Chu-Min Li, Jordi Coll, Shuolin Li, Djamal Habet, Felip Manyà, and Kun He. Maxcdcl in maxsat evaluation 2023. *MaxSAT Evaluation 2023*, pages 14–15, 2023.

[3] Chu-Min Li, Zhenxing Xu, Jordi Coll, Felip Manyà, Djamal Habet, and Kun He. Combining clause learning and branch and bound for maxsat. In *27th International Conference on Principles and Practice of Constraint Programming (CP 2021)*, 2021.

[4] Ruben Martins, Vasco M. Manquinho, and Inês Lynce. Open-WBO: A modular MaxSAT solver. In *Proceedings of SAT 2014*, volume 8561 of *LNCS*, pages 438–445. Springer, 2014.

noSAT-MaxSATv3

Ole Lübke
Institute for Software Systems
Hamburg University of Technology (TUHH)
Hamburg, Germany
ole.luebke@tuhh.de

***Abstract*—Using a SAT solver to solve MaxSAT is a well-established and efficient method. However, in resource-constrained computing environments, e.g., embedded systems, such algorithm designs can be hard to realize. With *noSAT-MaxSAT*, we explore alternatives that do not rely on an external, dedicated SAT solver. Compared to the previous version, the most notable changes in *noSAT-MaxSATv3* include an update of the base algorithm to NuWLS-2.0, new rules for restarting, the integration of a new Frequency Fitness Assignment-inspired variable selection heuristic in local optima, and the inclusion of fixed-precision integer arithmetic to increase precision in weight calculations without resorting to floating-point numbers.**

I. Introduction

Exact as well as anytime MaxSAT solvers usually include a SAT solver. For stochastic local search (SLS) algorithms, a SAT solver can provide a feasible model as a starting point [1]; linear search algorithms query a SAT solver iteratively; core-guided and implicit hitting set-based algorithms use a SAT solver to extract unsatisfiable subsets of the clauses of the formula (cores) [2]. Our goal is to develop an efficient MaxSAT solver that is suitable for deployment in resource-constrained computing environments. Consequently, the solver is developed under a set of constraints commonly found in embedded systems programming, outlined in section II, which practically rule out the inclusion of any complete SAT solving algorithm.

Compared to *noSAT-MaxSATv2* [3], in *noSAT-MaxSATv3* we removed the 2-SAT preprocessing step, which turned out to be effective only for very few instances; updated the base algorithm to the NuWLS-2.0 SLS algorithm [4]; introduced a new variable-selection heuristic for local optima inspired by Frequency Fitness Assignment (FFA) [5, 6]; improved precision in weight calculations using fixed-precision integer arithmetic; implemented new conditions for restarting. More details on these changes are provided in section III.

Finally, for comparison, we paired *noSAT-MaxSATv3* with the *CaDiCaL 1.9.5* SAT solver [7], resulting in the solver *noSAT-MaxSATv3-c*. It executes *CaDiCaL* once to obtain a feasible initial assignment for SLS (or determine unsatisfiability).

II. Requirements & Architecture

noSAT-MaxSAT is developed under the following requirements commonly found in programming embedded systems: It 1) is programmed in C; 2) is self-contained, i.e., it has no external dependencies (other than the C standard library); 3) does not allocate memory dynamically; 4) does not use floating-point operations; 5) does not contain unbounded loops, i.e., the maximum number of iterations of any loop is known and constant when entering it.

noSAT-MaxSAT is split into a library that fulfills these requirements, and an application that uses the library but is not bound by the above-mentioned restrictions to facilitate, e.g., parsing input instances of arbitrary size. Given the number of variables and clauses of a formula, the library can compute (an upper bound on) the amount of memory required for running the algorithm.

III. Implementation

Algorithm 1 noSAT-MaxSATv3

Require: Variable values are initialized (e.g., randomly)
1: $bestcost \leftarrow \infty; skip \leftarrow false; badtries \leftarrow 0$
2: **for** $try \leftarrow 1 \dots maxTries$ **do**
3: **if** $\neg skip$ **then**
4: INITVARS($badtries \geq maxBadtries$)
5: DECIMATION
6: INIT
7: **end if**
8: $skip \leftarrow false; impr \leftarrow false$
9: **for** $flip \leftarrow 1 \dots maxFlips$ **do**
10: $impr \leftarrow current\ cost < bestcost$
11: $skip \leftarrow skip \vee impr$
12: $v \leftarrow$ SELECTVARIABLE
13: FLIPVARIABLE(v)
14: **end for**
15: $impr \leftarrow current\ cost < bestcost$
16: $skip \leftarrow skip \vee impr$
17: $badtries \leftarrow$ **if** $impr$ **then** 0 **else** $badtries + 1$
18: **end for**

Algorithm 1 shows how NuWLS-2.0 [4] is implemented in *noSAT-MaxSATv3*, highlighting the new restart rules. In the following, we elaborate on these rules, the new variable selection heuristic, and the use of fixed-precision integer arithmetic.

A. New Restart Rules

In line 3 of Algorithm 1, the initialization block is only entered when no cost-improving assignment has been found

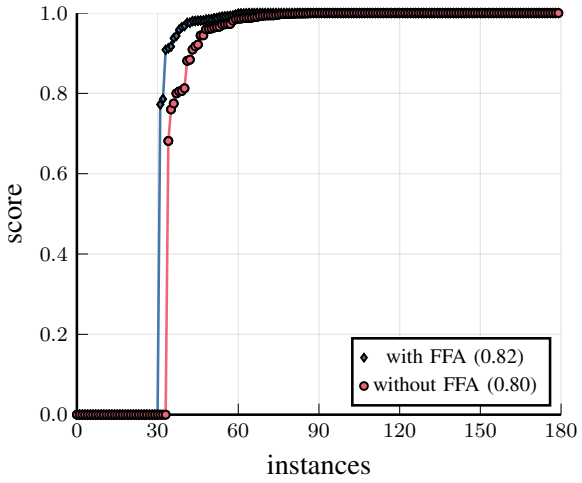


Fig. 1. Comparison of *noSAT-MaxSATv3* with and without the FFA-based variable selection heuristic. $score(s,i) = \frac{1+\min_{z\in\{\text{with FFA,without FFA}\}} o(z,i)}{1+o(s,i)}$, where $o(s,i)$ denotes the best objective value found by solver s on instance i within 60 seconds. The benchmark instances have been selected randomly from the incomplete/anytime weighted tracks of the MaxSAT Evaluations 2020 – 2023.

during the last iteration of the outer loop. If so, in line 4, a random variable assignment is generated when no cost-improving assignment has been found for $maxBadtries$ iterations of the outer loop. When there are any falsified hard clauses, Algorithm 1 is executed on all hard clauses to find a feasible initial assignment. In line 5, decimation [8] is used to improve the assignment.

The original implementation of NuWLS-2.0 [4] effectively resets $flip$ to 0 whenever a better assignment is found, to continue searching in the same, promising direction. Resetting loop counters is not allowed in *noSAT-MaxSAT* due to requirement 5. Instead, the $skip$ flag is set to skip the initialization procedures for the next try and continue searching in the same direction. When no improvement was found for $maxBadtries$ tries, Algorithm 1 restarts with a random assignment to explore different areas of the search space.

B. Variable Selection Inspired by FFA

Frequency Fitness Assignment (FFA) is a heuristic used in evolutionary algorithms which prefers individuals with rarely-encountered fitness values over individuals with high fitness values [5]. In SLS algorithms for MaxSAT such as NuWLS, usually the variable with the highest *score* is selected for flipping, where the score indicates how much the total cost of the current assignment improves when the variable is flipped. With FFA we would select the variable with the least-frequently encountered score instead. However, maintaining a dictionary of score values and their frequencies can impose a significant overhead. In contrast, an array of variable flip counts can be maintained with negligible, constant overhead, and selecting rarely-flipped variables over variables with a high score can actually lead to discovering better solutions [6]. Therefore, whenever there are no variables with a positive score (i.e., a local optimum is reached), the clause weights are updated, and a falsified clause is selected randomly (preferring hard clauses) as in NuWLS-2.0 [4]. Then, instead of selecting the variable with the least bad score, we select the least frequently flipped variable from that clause. Figure 1 shows that *noSAT-MaxSATv3* performs slightly better with the FFA-based heuristic.

C. Fixed-Precision Integer Arithmetic

The weighting scheme of NuWLS-2.0 [4] requires calculating the average weight of all soft clauses. In contrast to the original implementation, we cannot use floating-point numbers due to requirement 4. However, we found that the loss of precision under usual truncating integer arithmetic can severely degrade the performance of the solver. As a remedy, we scale all values involved in weight and score computations by left-shifting by x bits (multiplying with 2^x) where $x = 64 - b_{reserved} - b_{weightsum}$, $b_{reserved}$ is a number of reserved bits to prevent overflows (currently 32), and $b_{weightsum}$ is the number of bits required to store the sum of all weights.

References

[1] Zhendong Lei and Shaowai Cai. “SATLike-c: Solver Description”. In: *MaxSAT Evaluation 2020: Solver and Benchmark Descriptions*. 2020, p. 23.

[2] Fahiem Bacchus, Matti Järvisalo, and Ruben Martins. “Maximum Satisfiability”. In: *Handbook of Satisfiability*. 2nd ed. Vol. 336. Front. Artif. Intell. Appl. 2021, pp. 929–991. DOI: 10.3233/FAIA201008.

[3] Ole Lübke and Sibylle Schupp. “noSAT-MaxSATv2”. In: *MaxSAT Evaluation 2023: Solver and Benchmark Descriptions*. 2023, pp. 27–28.

[4] Yi Chu, Shaowei Cai, and Chuan Luo. “NuWLS-c-2023: Solver Description”. In: *MaxSAT Evaluation 2023: Solver and Benchmark Descriptions*. 2023, pp. 23–24.

[5] Thomas Weise, Mingxu Wan, Pu Wang, Ke Tang, Alexandre Devert, and Xin Yao. “Frequency Fitness Assignment”. In: *IEEE Trans. Evol. Comp.* 18.2 (2014), pp. 226–243. DOI: 10.1109/TEVC.2013.2251885.

[6] Thorge Becker. “Application of Frequency Fitness Assignment for Solving MaxSAT Problems”. Project Thesis. Hamburg University of Technology, 2023. URL: https://www.sts.tuhh.de/pw-and-m-theses/2023/becker23.pdf.

[7] Armin Biere, Katalin Fazekas, Mathias Fleury, and Maximillian Heisinger. “CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling Entering the SAT Competition 2020”. In: *Proc. of SAT Competition 2020 – Solver and Benchmark Descriptions*. 2020, pp. 51–53.

[8] Shaowei Cai, Chuan Luo, and Haochen Zhang. “From Decimation to Local Search and Back: A New Approach to MaxSAT”. In: *Proc. 26th Int. Jt. Conf. Artif. Intell.* 2017, pp. 571–577. DOI: 10.24963/ijcai.2017/80.

TT-Open-WBO-Inc-24: an Anytime MaxSAT Solver Entering MSE'24

Alexander Nadel
Data and Decision Sciences, Technion, Haifa, Israel
Email: alexander.nadel@cs.tau.ac.il

***Abstract*—This document describes the solver TT-Open-WBO-Inc-24, submitted to the four incomplete tracks of MaxSAT Evaluation 2024. TT-Open-WBO-Inc-24 is the 2024 version of our solver TT-Open-WBO-Inc [8], itself based on Open-WBO-Inc [4]. The main innovation in TT-Open-WBO-Inc-24 is the integration of the local search component from NuWLS-c-2023 [3].**

I. Introduction

TT-Open-WBO-Inc [8] is our anytime MaxSAT solver, based on Open-WBO-Inc [4]. Mostly similarly to the previous year's version [10], TT-Open-WBO-Inc-24 combines the following algorithms:

1) NuWLS-c-2023 local search [3] for preprocessing (the only significant change from the previous year's version, based on the 2022 version of NuWLS-c).
2) The unweighted component uses Mrs. Beaver [6], enhanced by the following two heuristics from Sect. 4.1 in [5]: global stopping condition for OBV-BS and size-based switching to complete part.
3) The weighted component uses BMO-based clustering [4].
4) The Polosat SAT-based local search algorithm [7] replaces the regular SAT invocations in both the unweighted and weighted components.

We adjusted some of the low-level parameters of the aforementioned algorithms to the benchmarks from the latest MaxSAT Evaluation. Additionally, we fixed several bugs unearthed by running the regression, provided by MaxSAT Evaluation 2024 organizers. As a result, TT-Open-WBO-Inc now works correctly on unsatisfiable instances and instances without any soft clauses.

We submitted two versions of TT-Open-WBO-Inc-24, the difference being the underlying SAT solver:

1) TT-Open-WBO-Inc-24(I): with IntelSAT [9].
2) TT-Open-WBO-Inc-24(G): with Glucose 4.1 [1].

References

[1] G. Audemard and L. Simon. On the Glucose SAT solver. *Int. J. Artif. Intell. Tools*, 27(1):1840001:1–1840001:25, 2018.
[2] J. Berg, M. Järvisalo, R. Martins, and A. Niskanen, editors. *MaxSAT Evaluation 2023: Solver and Benchmark Descriptions*. Department of Computer Science Series of Publications B. Department of Computer Science, University of Helsinki, Finland, 2023.
[3] Y. Chu, S. Cai, and C. Luo. Nuwls-c-2023: Solver description. In Berg et al. [2].
[4] S. Joshi, P. Kumar, S. Rao, and R. Martins. Open-wbo-inc: Approximation strategies for incomplete weighted maxsat. *J. Satisf. Boolean Model. Comput.*, 11(1):73–97, 2019.
[5] A. Nadel. Anytime weighted MaxSAT with improved polarity selection and bit-vector optimization. In *FMCAD 2019*, pages 193–202.
[6] A. Nadel. Solving MaxSAT with bit-vector optimization. In *SAT 2018*, pages 54–72, 2018.
[7] A. Nadel. On optimizing a generic function in SAT. In *2020 Formal Methods in Computer Aided Design, FMCAD 2020, Haifa, Israel, September 21-24, 2020*, pages 205–213. IEEE, 2020.
[8] A. Nadel. Polarity and variable selection heuristics for SAT-based anytime MaxSAT. *J. Satisf. Boolean Model. Comput.*, 12(1):17–22, 2020.
[9] A. Nadel. Introducing Intel(R) SAT solver. In K. S. Meel and O. Strichman, editors, *25th International Conference on Theory and Applications of Satisfiability Testing, SAT 2022, August 2-5, 2022, Haifa, Israel*, volume 236 of *LIPIcs*, pages 8:1–8:23. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.
[10] A. Nadel. TT-Open-WBO-Inc-23: an Anytime MaxSAT Solver Entering MSE'23. In Berg et al. [2].

NuWLS-c-IBR: Solver Description

Menghua Jiang[1] and Yin Chen[1,2]
[1] School of Computer Science, South China Normal University, Guangzhou, China
[2] School of Artificial Intelligence, South China Normal University, Guangzhou, China
{jiangmenghua}@m.scnu.edu.cn, {ychen}@scnu.edu.cn

***Abstract*—This document describes the solver NuWLS-c-IBR, submitted to both the weighted and unweighted anytime tracks of MaxSAT Evaluation 2024.**

I. Introduction

Our NuWLS-c-IBR solver is an improved version of the NuWLS-c-2023 [1] solver that participated in MaxSAT Evaluation 2023. Like SATLike-c [2] and NuWLS-c [3], NuWLS-c-IBR has two engines. One is our proposed stochastic local search (SLS) algorithm NuWLS-IBR, which introduces an improved weighting scheme for NuWLS-2.0. The other is the SAT-based algorithm TT-Open-WBO-Inc.

II. NuWLS-IBR

A. Preliminaries

For each clause c, we use $w(c)$ to denote the weight of clause c. For each soft clause c, we use $w_{\text{org}}(c)$ to denote the original weight of c, which is given in the instance. $avg_{\text{softw}}(c)$ denotes the average original soft clause weights.

B. Rate-Weighting

In the literature [4], we note that a dynamic weighting method that increases the weight of a soft clause proportionally based on its current weight can guide the search direction and help overcome local optima. Therefore, we have designed the following weighting scheme, called **Rate-Weighting**.

The settings of NuWLS-c-IBR is as follows:

- *h_inc* := 1 for unweighted instances, and *h_inc* := 5 for weighted instances.
- *s_inc* := 1 for unweighted instances, and *s_inc* := 6 for weighted instances.
- δ := 1.001.

At the beginning of each round of the local search process, the Rate-Weighting scheme initializes each clause weight as follows:

- For each hard clause c, we set $w(c)$:= 1.
- For each soft clause c, we set $w(c)$:= 1 for unweighted instances, and $w(c)$:= 0 for weighted instances.

When the local search algorithm encounters a local optimum, the clause weights are updated as follows:

- For hard clauses: for each falsified hard clause c, $w(c) := w(c) + h\_inc$.
- For soft clauses: for each falsified soft clause c, $w(c) := w(c) + s\_inc$ for unweighted instances, and $w(c) := \delta \times (w(c) + \frac{w_{\text{org}}(c)}{(avg_{\text{softw}}(c)})$ for weighted instances.

Algorithm 1: Rate-Weighting

Input: A (W)PMS instance F, The best solution found so far α^*

```
for each falsified hard clause c do
    w(c) := w(c) + h_inc;
if α* is feasible then
    if is WPMS instance then
        for each falsified soft clause c do
            w(c) := w(c) + s_inc;
    else
        for each falsified soft clause c do
            w(c) := δ × (w(c) + w_org(c) / avg_softw(c));
return;
```

The pseudo-code of Rate-Weighting is outlined in algorithm 1. Based on the Rate-Weighting scheme and the dynamic local search framework, we develop a new SLS algorithm named NuWLS-IBR.

III. The Hybrid Solver: NuWLS-c-IBR

We combine NuWLS-IBR with the state-of-the-art SAT-based solver TT-Open-WBO-Inc [5], leading to the hybrid solver NuWLS-c-IBR. The framework of NuWLS-c-IBR is similar to NuWLS-c-2023 [1].

References

[1] Chu Y, Cai S, Luo C. NuWLS-c-2023: Solver description. MaxSAT Evaluation 2023, 2023: 23.
[2] Lei Z, Cai S, Geng F, et al. SATLike-c: Solver description. MaxSAT Evaluation, 2021, 2021: 19.
[3] Chu Y, Cai S, Lei Z, et al. Nuwls-c: Solver description. MaxSAT Evaluation, 2022, 2022: 28.
[4] Zheng J, Chen Z, Li C M, et al. Rethinking the Soft Conflict Pseudo Boolean Constraint on MaxSAT Local Search Solvers. arXiv preprint arXiv:2401.10589, 2024.
[5] Nadel A. Anytime weighted MaxSAT with improved polarity selection and bit-vector optimization//2019 Formal Methods in Computer Aided Design (FMCAD). IEEE, 2019: 193-202.

Combining BandMaxSAT and FPS with SPB-MaxSAT-c

Mingming Jin[1,2] Kun He[1,2] Jiongzhi Zheng[1,2] Jinghui Xue[1,2] Zhuo Chen[1,2]
[1]School of Computer Science and Technology, Huazhong University of Science and Technology, China
[2]Hopcroft Center on Computing Science, Huazhong University of Science and Technology, China
{mingmingk,brooklet60,jzzheng,jh_xue,ciaozer}@hust.edu.cn

***Abstract*—This document describes our MaxSAT solvers SPB-MaxSAT-c, SPB-MaxSAT-c-Band and SPB-MaxSAT-c-FPS submitted to the MSE 2024. All of the three solvers are submitted to both the weighted and unweighted incomplete tracks.**

I. Introduction

Algorithms for MaxSAT can be categorized into complete and incomplete solvers, depending on their ability to provide optimality guarantees.

An important technique in MaxSAT complete-solving is the Soft conflict Pseudo Boolean (SPB) constrain. A Pseudo Boolean (PB) constraint [1] is a particular type of linear constraint in the form of $\sum_{i=1}^{n} q_i x_i \ \# \ K$, where $\# \in \{<, \leq, =, \geq, >\}$ and can be normalized to $<$, q_i and K are integer constants, and x_i are 0/1 variables, $i \in \{1, ..., n\}$. In many complete MaxSAT solvers, once a better solution is found, a PB (or Cardinality for unweighted problem) constraint is added naturally to prohibit the procedure from finding solutions that are no better than the current best.

On the other hand, incomplete algorithms for MaxSAT mainly focus on the local search approach.Recent research in this domain has concentrated on refining clause weighting schemes, such as SATLike [2] and NuWLS [3]. The anytime algorithms based on them, i.e., SATLike-c and NuWLS-c, combine them with a famous SAT-based solver, TT-Open-WBO-Inc [4] and have achieved excellent rankings in the previous MaxSAT Evaluations.

Based on Nuwls-c-2023, we integrate SPB constraints into the clause weighting system and develop a new local search algorithm SPB-MaxSAT-c [5]. Then we combine BandMaxSAT [6] and FPS [7] with the SPB-MaxSAT-c solver, resulting in SPB-MaxSAT-c-Band and SPB-MaxSAT-c-FPS, respectively.

II. SPB-MaxSAT

SPB-MaxSAT [5] associate each hard clause with a dynamic weight and add a SPB constraint to the MaxSAT formula with a dynamic weight. When the algorithm hits a local optimum, the dynamic weights of the falsified hard clauses and SPB constraint are increased. Additionally, when a better solution is found through local search, the SPB constraint is updated. This method guides the search directions and helps the algorithm find better solutions without restricting the search space.

SPB Constraint

Algorithm 1: SPB-Weighting

Input: MaxSAT instance $\mathcal{F}$, current solution A, hard clause dynamic weight increment h_{inc}, increment proportion δ

for *each clause* $c \in Hard(\mathcal{F})$ *falsified by* A **do**
  $w_h(c) \leftarrow w_h(c) + h_{inc}$;
if SPB *is falsified by* A **then**
  $w(SPB) \leftarrow \delta \cdot (w(SPB) + 1)$;

Suppose A^* is the best solution found so far during the local search. Finding solutions no better than A^* is meaningless, so an SPB constraint, denoted as SPB, is naturally added as follows.

$$SPB : \sum_{c \in Soft(\mathcal{F})} w_s(c) \cdot unsat(c) < cost(A^*), \qquad (1)$$

where $unsat(c)$ equals 1 if clause c is falsified by the current solution A. Denote the left part of the SPB as $obj(A)$.

Scoring Function

In the local search MaxSAT algorithms, the scoring function $score(v)$ of a variable v is commonly used to evaluate the benefit of flipping v in the current solution.

Suppose A' is the solution obtained by flipping variable v in A. $SPBscore(v)$ is defined to to evaluate the influence of flipping v in A to the SPB constrain.

$$SPBscore(v) = w(SPB) \cdot (obj(A) - obj(A')). \qquad (2)$$

The scoring function $score(v)$ is defined as follows:

$$score(v) = hscore(v) + SPBscore(v). \qquad (3)$$

where $hscore(v)$ is the decrement of the total dynamic weight of falsified hard clauses caused by flipping v in A.

Adaptive SPB Weighting

The proposed SPB-Weighting function is shown in Algorithm 1. The function is used to update the dynamic weights once the algorithm falls into a local optimum.

III. BandMaxSAT

BandMaxSAT [6] integrates a multi-armed bandit with the soft clauses, where each arm corresponds to a soft clause. This approach employs the bandit model to strategically choose the search direction, aiding in the evasion of feasible local optima.

IV. FPS

The Farsighted Probabilistic Sampling (FPS) method [7] combines the look-ahead strategy with the probabilistic sampling strategy in an effective way. Upon encountering a local optimum, FPS first randomly samples sc_num (10 by default) falsified clauses, then proceeds to perform a look-ahead from a randomly chosen variable within each sampled clause. This step evaluates the potential improvement achievable by flipping a pair of variables in the current solution. If this attempt proves unsuccessful, FPS then selects the optimal variable or pair of variables among the sampled set to flip.

With the help of the look-ahead strategy, FPS can improve the local optima for the single flipping mechanism, so as to find higher-quality solutions. Meanwhile, the probabilistic sampling strategy contributes to the algorithm's efficiency enhancement, augmenting its overall performance.

REFERENCES

[1] Li C M, Xu Z, Coll J, et al. "Combining clause learning and branch and bound for MaxSAT," In Proceedings of the 27th International Conference on Principles and Practice of Constraint Programming,volume 210, pages 38:1–38:18, 2021.

[2] S. Cai, Z. Lei, "Old techniques in new ways: Clause weighting, unit propagation and hybridization for maximum satisfiability," Artificial Intelligence, 287: 103354, 2020.

[3] Chu Y, Cai S, Luo C. "NuWLS: Improving local search for (weighted) partial MaxSAT by new weighting techniques," AAAI 2023, 37(4): 3915-3923.

[4] N. Alexander, "Anytime weighted maxsat withimproved polarity selection and bit-vector optimization," FMCAD 2019: 193–202.

[5] Zheng J, Chen Z, Li C M, et al. "Rethinking the Soft Conflict Pseudo Boolean Constraint on MaxSAT Local Search Solvers," arXiv preprint arXiv:2401.10589, 2024.

[6] J. Zheng, K. He, J. Zhou, Y. Jin, C. M. Li, F. Manyà, "BandMaxSAT: A Local Search MaxSAT Solver with Multi-armed Bandit," IJCAI 2022: 1901-1907.

[7] J. Zheng, J. Zhou, K. He, "Farsighted Probabilistic Sampling based Local Search for (Weighted) Partial MaxSAT," AAAI 2023.

CASHWMaxSAT-DisjCad: Solver Description

Shiwei Pan[1,2], Yiyuan Wang[1,2,*], Shaowei Cai[3,4,*], Jiangnan Li[1,2], Wenbo Zhu[1,2], Minghao Yin[1,2]
[1]School of Computer Science and Information Technology, Northeast Normal University, China
[2]Key Laboratory of Applied Statistics of MOE, Northeast Normal University, Changchun, China
[3]State Key Laboratory of Computer Science, Institute of Software, Chinese Academy of Sciences, Beijing, China
[4]School of Computer Science and Technology, University of Chinese Academy of Sciences, China
[*]corrsponding author
yiyuanwangjlu@126.com, caisw@ios.ac.cn,
{pansw779, lijn101, zhuwenbo, ymh}@nenu.edu.cn

***Abstract*—This document describes the MaxSAT solver CASHWMaxSAT-DisjCad, submitted to the complete tracks (include unweighted and weighted track) of MaxSAT Evaluation 2023.**

I. Introduction

We developed an improved complete MaxSAT solver called CASHWMaxSAT-DisjCad based on the latest version UWrMaxSat 1.5.4 [1], SCIP 8.1.0 [2] and CASHWMaxSAT-CorePuls 2023 [3]. In addition, CASHWMaxSAT-DisjCad used an unsatisfiable-core-based OLL procedure [4]–[7]. In this work, we propose two novel ideas to improve the last year's version of CASHWMaxSAT-CorePlus [3]. Based on CASHWMaxSAT-CorePlus, we optimize the parameter settings of SCIP, use an underlying SAT solver CaDiCaL[1] [8], and develop a new algorithm called CASHWMaxSAT-DisjCad.

- In [9], Extracting multiple disjoint cores in each iteration of the IHS algorithm can enhance the performance of solving MaxSAT. We have incorporated this idea into our solver. Initially, we obtain an unsatisfiable core using a SAT solver. Then, we remove this unsatisfiable core from the original assumptions and solve again using the SAT solver to get new disjoint unsatisfiable cores. This process is repeated until the used SAT solver returns SAT.
- We exclude some soft clauses that can be optimized by the Boolean multilevel optimization. For the remaining soft clauses, we calculate the standard deviation of their weight values. If the obtained standard deviation is less than the average value of their weight values, we add all the assumptions into the SAT solver.

II. Future work

First, we could use a simplified version of MaxSAT local search solvers to improve the satisfied solution. Second, we could try to design a novel selection way for selecting an unsatisfiable-core on weighted cases.

[1]https://github.com/arminbiere/cadical

References

[1] M. Piotrów, "Uwrmaxsat in maxsat evaluation 2021," *MaxSAT Evaluation 2021*, p. 17, 2021.
[2] K. Bestuzheva, M. Besançon, W.-K. Chen, A. Chmiela, T. Donkiewicz, J. van Doornmalen, L. Eifler, O. Gaul, G. Gamrath, A. Gleixner *et al.*, "The scip optimization suite 8.0," *arXiv preprint arXiv:2112.08872*, 2021.
[3] S. Pan, Y. Wang, Z. Lei, S. Cai, S. Wang, and M. Yin, "Cashwmaxsat-coreplus: Solver description," *MaxSAT Evaluation 2023*, p. 1, 2024.
[4] B. Andres, B. Kaufmann, O. Matheis, and T. Schaub, "Unsatisfiability-based optimization in clasp," in *Technical Communications of the 28th International Conference on Logic Programming (ICLP'12)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2012.
[5] A. Morgado, F. Heras, M. Liffiton, J. Planes, and J. Marques-Silva, "Iterative and core-guided maxsat solving: A survey and assessment," *Constraints*, vol. 18, no. 4, pp. 478–534, 2013.
[6] A. Morgado, C. Dodaro, and J. Marques-Silva, "Core-guided maxsat with soft cardinality constraints," in *International Conference on Principles and Practice of Constraint Programming*. Springer, 2014, pp. 564–573.
[7] A. Ignatiev, A. Morgado, and J. Marques-Silva, "Rc2: an efficient maxsat solver," *Journal on Satisfiability, Boolean Modeling and Computation*, vol. 11, no. 1, pp. 53–64, 2019.
[8] A. Biere, K. Fazekas, M. Fleury, and M. Heisinger, "CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020," in *Proc. of SAT Competition 2020 – Solver and Benchmark Descriptions*, ser. Department of Computer Science Report Series B, vol. B-2020-1. University of Helsinki, 2020, pp. 51–53.
[9] P. Saikko, "Re-implementing and extending a hybrid sat-ip approach to maximum satisfiability," Ph.D. dissertation, Master's thesis, University of Helsinki, 2015.

Pacose: An Iterative SAT-based MaxSAT Solver

Tobias Paxian, Bernd Becker
Albert-Ludwigs-Universität Freiburg
Georges-Köhler-Allee 051
79110 Freiburg, Germany
{paxiant|becker}@informatik.uni-freiburg.de

I. Overview

Pacose is a SAT-based MaxSAT solver, using two incremental CNF encodings, a binary adder [1] and the Dynamic Polynomial Watchdog (DPW) [2], for Pseudo-Boolean (PB) constraints. It is an extension of QMaxSAT 2017 [3], based on Glucose 4.2.1 [4] SAT solver. It uses a Boolean Multilevel Optimization (BMO) pre- / inprocessing method to simplify the instances. Additionally a trimming method is applied to cut off unsatisfiable soft clauses and find a good initial satisfiable weight to reduce the size of the encoding.

II. Pre- / Inprocessing

The 2023 version of Pacose contains the preprocessing tool maxpre version 2 [5] and performs our own preprocessing routines Generalized Boolean Multilevel Optimization (GBMO) and TrimMaxSAT [6].

MaxPre version 1 [7], has already been utilized by several MaxSAT solvers in past competitions. Preprocessing has gained increasing importance in recent years, with many top solvers from previous years adopting MaxPre and other techniques to achieve favorable results. The successful outcomes reported in [5] have further persuaded us to incorporate version 2 of this preprocessor into our own implementation.

We generalized the plain variant of Boolean Multilevel Optimization to work with arbitrary weights and split additional instances with that. Further we implemented a greedy algorithm TrimMaxSAT to remove never satisfiable instances and getting first upper / lower bounds.

III. Encoding and Algorithm

Our DPW encoding is based on the Polynomial Watchdog (PW) encoding [8], which uses totalizer networks [9]. Essentially the DPW encoding employs multiple totalizer networks to perform a binary addition with carry on the sorted outputs. A special algorithm to solve these instances incremental is presented in [2].

Additionally the adder network [1] is used which has a linear complexity in encoding size in contrast to at least $\mathcal{O}(n^2)$ for the DPW sorting network. With the adder network many complementary instances to the DPW encoding can be solved and therefore it is well suited, to be chosen, together with DPW by a heuristic, as described in the following chapter. The algorithm and encoding are partly adapted and inspired from QMaxSAT.

IV. Heuristics

Pacose uses straightforward heuristics based on available MaxSAT benchmarks. All heuristics are based on the number of soft clauses and the overall sum of soft weights.

- *Encoding:* The DPW encoding empirically works best if the average weight for soft clauses is small, or the overall sum of soft weights is huge (bigger than 80 billion). For other instances the binary adder is chosen.
- *Trimming:* As for instances with only a few soft clauses the trimming preprocessing algorithm is not effective, it is only used if the benchmark contains at least a certain amount of soft clauses.
- *Compression Rate:* For benchmarks with only a few soft clauses, the encoding is smaller and additional clauses can be added. Therefore, the binary adder encoding can solve overall more benchmarks if the compression rate is chosen accordingly.

References

[1] J. P. Warners, "A linear-time transformation of linear inequalities into conjunctive normal form," *Information Processing Letters*, vol. 68, no. 2, pp. 63–69, 1998.

[2] T. Paxian, S. Reimer, and B. Becker, "Dynamic polynomial watchdog encoding for solving weighted MaxSAT," in *International Conference on Theory and Applications of Satisfiability Testing*. Springer, 2018, pp. 37–53.

[3] M. Koshimura, T. Zhang, H. Fujita, and R. Hasegawa, "QMaxSAT: A partial Max-SAT solver system description," *Journal on Satisfiability, Boolean Modeling and Computation*, vol. 8, pp. 95–100, 2012.

[4] G. Audemard and L. Simon, "On the glucose SAT solver," *International Journal on Artificial Intelligence Tools*, vol. 27, no. 01, p. 1840001, 2018.

[5] H. Ihalainen, J. Berg, and M. Järvisalo, "Clause redundancy and preprocessing in maximum satisfiability," in *Automated Reasoning: 11th International Joint Conference, IJCAR 2022, Haifa, Israel, August 8–10, 2022, Proceedings*. Springer, 2022, pp. 75–94.

[6] T. Paxian, P. Raiola, and B. Becker, "On preprocessing for weighted MaxSAT," in *Verification, Model Checking, and Abstract Interpretation: 22nd International Conference, VMCAI 2021, Copenhagen, Denmark, January 17–19, 2021, Proceedings 22*. Springer, 2021, pp. 556–577.

[7] T. Korhonen, J. Berg, P. Saikko, and M. Järvisalo, "Maxpre: an extended maxsat preprocessor," in *Theory and Applications of Satisfiability Testing–SAT 2017: 20th International Conference, Melbourne, VIC, Australia, August 28–September 1, 2017, Proceedings 20*. Springer, 2017, pp. 449–456.

[8] O. Bailleux, Y. Boufkhad, and O. Roussel, "New encodings of pseudo-boolean constraints into CNF," in *International Conference on Theory and Applications of Satisfiability Testing*. Springer, 2009, pp. 181–194.

[9] O. Bailleux and Y. Boufkhad, "Efficient CNF encoding of Boolean cardinality constraints," in *Principles and Practice of Constraint Programming–CP 2003*. Springer, 2003, pp. 108–122.

UWrMaxSat Entering the MaxSAT Evaluation 2024

Marek Piotrów
Institute of Computer Science, University of Wrocław
Wrocław, Poland
marek.piotrow@cs.uni.wroc.pl

***Abstract*—UWrMaxSat is a complete solver for partial weighted MaxSAT instances and pseudo-Boolean ones. It can be also characterized as an anytime solver, since it outputs the best known solution, whenever its run is interrupted. It needs a SAT solver as an oracle and can be used with a few modern solvers, from which COMiniSatPS by Chanseok Oh (2016) has been selected as a default one. Several solving strategies have been implemented in it (selected by parameters) but the default one is an UNSAT-based core-guided OLL procedure, where a sorter-based encoding is applied to translate cardinality constraints into CNF. If this strategy is not successful for a selected time, it is changed to a SAT/UNSAT-based binary search. For relatively small instances, beside the MaxSAT solver, the SCIP mixed-integer-programming solver is used as an independent advanced tool with its, quite different, solving techniques. This paper describes new elements in UWrMaxSat version 1.6, which is submitted to the MaxSAT Evaluation 2024. They include (1) a closer integration with SCIP solver for mixed integer programming, and (2) an implementation of the general Boolean multi-level optimization in the SAT/UNSAT strategy.**

***Index Terms*—MaxSAT-solver, UWrMaxSat, COMiniSatPS, SCIP, sorter-based encoding, core-guided, complete solver**

I. Description

It is the fifth time when UWrMaxSat is participating in MaxSAT Evaluation (MSE). Its description can be found in the series of publications [6], [7], [9], [10] and a more complete presentation in [8]. See there for the main features added to the solver in previous years.

The new version is denoted by 1.6 and contains two new extensions. Firstly, the cooperation with SCIP solver [2] is more involved, especially when both solvers has to run in the same thread (as it is required by the competition rules). Secondly, the GBMO (that is, general Boolean multi-level optimization, see [5]) splitting points are now also used in the UNSAT/SAT binary-search solving phase.

The results of recent MSE competitions showed that extending the standard set of MaxSAT techniques with the mixed-integer programming (MIP) methods implemented in a MIP solver can increase the number of solved instances. The SCIP optimizing suite was chosen as an extension of UWrMaxSat in its version 1.4. In the standard configuration both solvers run in separate threads and the first which solves the instance is a winner and reports its result. CNF clauses are preprocessed by COMiniSatPS SAT solver before they are converted into Boolean linear inequations for SCIP. Earlier, the MaxPre preprocessor [4] can further reduce the numbers of variables/clauses in an instance and it can be easily switch on by an option of UWrMaxSat. In the single thread, the SCIP is run first for a predefined number of second and if it is not successful, UWrMaxSat tries to solve the instance. In such a case, the partial results of SCIP were discarded.

In the present version of UWrMaxSat, the start of SCIP solver can be delayed for a certain number of seconds and UWrMaxSat tries to solve an instance in meantime. If it is unsuccessful, its lower and upper bounds on the weight of an optimal solution are given as an additional input to SCIP. When the latter timeouts, its lower/upper bounds and the best solution are compared with the previous results of UWrMaxSat and can improve them. Then the solving process of the MaxSAT solver is continued. In addition, if the relative gap between the bound of SCIP is less than 10 percent, the time assigned to SCIP is extended and it resumes its computation.

Finding GBMO splitting points is computationally expensive, therefore UWrMaxSat tries to find some of them within limited resources. When found, they were previously used only in the process of stratification [1]. Now, the SAT/UNSAT binary search is done independently in each region between consecutive splitting points. In this way, encodings of goal linear functions are less costly.

Finally, it is the first time when the MaxPre is switched on in the solver submitted to the unweighted exact track with the time limit of 60s for its techniques. Generally, preprocessing is an important part of an system for solving MaxSAT problems. An interesting question is whether an output of the SCIP preprocessing can be effectively used by a MaxSAT solver.

References

[1] Carlos Ansótegui, Maria Luisa Bonet, Joel Gabàs, and Jordi Levy. Improving sat-based weighted maxsat solvers. In Michela Milano, editor, *Proc. CP*, pages 86–101. Springer, 2012.

[2] Ksenia Bestuzheva, Mathieu Besançon, Wei-Kun Chen, Antonia Chmiela, Tim Donkiewicz, Jasper van Doornmalen, Leon Eifler, Oliver Gaul, Gerald Gamrath, Ambros Gleixner, Leona Gottwald, Christoph Graczyk, Katrin Halbig, Alexander Hoen, Christopher Hojny, Rolf van der Hulst, Thorsten Koch, Marco Lübbecke, Stephen J. Maher, Frederic Matter, Erik Mühmer, Benjamin Müller, Marc E. Pfetsch, Daniel Rehfeldt, Steffan Schlein, Franziska Schlösser, Felipe Serrano, Yuji Shinano, Boro Sofranac, Mark Turner, Stefan Vigerske, Fabian Wegscheider, Philipp Wellner, Dieter Weninger, and Jakob Witzig. The SCIP Optimization Suite 8.0. ZIB-Report 21-41, Zuse Institute Berlin, December 2021.

[3] Michał Karpiński and Marek Piotrów. Reusing comparator networks in pseudo-boolean encodings. *arXiv preprint arXiv:2205.04129*, 2022.

[4] Tuukka Korhonen, Jeremias Berg, Paul Saikko, and Matti Järvisalo. MaxPre: An extended MaxSAT preprocessor. In Serge Gaspers and Toby Walsh, editors, *Proc. SAT*, volume 10491 of *LNCS*, pages 449–456. Springer, 2017.

[5] Tobias Paxian, Pascal Raiola, and Bernd Becker. On preprocessing for weighted maxsat. In Fritz Henglein, Sharon Shoham, and Yakir Vizel, editors, *Verification, Model Checking, and Abstract Interpretation*, pages 556–577, Cham, 2021. Springer International Publishing.

[6] Marek Piotrów. Uwrmaxsat - a new minisat+-based solver in maxsat evaluation 2019. In Fahiem Bacchus, Matti Järvisalo, and Ruben Martins, editors, *MaxSAT Evaluation 2019 : Solver and Benchmark Descriptions*, Department of Computer Science Report Series B-2019-2, pages 11–12, 2019.

[7] Marek Piotrów. Uwrmaxsat - an efficient solver in maxsat evaluation 2020. In Fahiem Bacchus, Jeremias Berg, Matti Järvisalo, and Ruben Martins, editors, *MaxSAT Evaluation 2020 : Solver and Benchmark Descriptions*, Department of Computer Science Report Series B-2020-2, pages 34–35, 2020.

[8] Marek Piotrów. Uwrmaxsat: Efficient solver for maxsat and pseudo-boolean problems. In *2020 IEEE 32nd International Conference on Tools with Artificial Intelligence (ICTAI)*, pages 132–136, 2020.

[9] Marek Piotrów. Uwrmaxsat in maxsat evaluation 2021. In Fahiem Bacchus, Jeremias Berg, Matti Järvisalo, and Ruben Martins, editors, *MaxSAT Evaluation 2021 : Solver and Benchmark Descriptions*, Department of Computer Science Report Series B-2021-2, pages 17–18, 2021.

[10] Marek Piotrów. Uwrmaxsat in maxsat evaluation 2022. In Fahiem Bacchus, Jeremias Berg, Matti Järvisalo, Ruben Martins, and Andreas Niskanen, editors, *MaxSAT Evaluation 2022: Solver and Benchmark Descriptions*, Department of Computer Science Series of Publications B, vol. B-2022-2, pages 21–22, 2022.

BENCHMARKS

MaxSAT Evaluation 2024: Solver and Benchmark Descriptions, volume B-2024-2 of Department of Computer Science Series of Publications B, University of Helsinki 2024.

Preprocessing for Nonmonotonic c-Inference via Partial MaxSAT: Benchmarks

Martin von Berg, Arthur Sanin, Aron Spang, Christoph Beierle
Knowledge-Based Systems
Faculty of Mathematics and Computer Science
FernUniversität in Hagen
58084 Hagen, Germany

***Abstract*—Skeptical c-inference is an entailment relation that exhibits desirable properties for nonmonotonic reasoning from conditional knowledge bases. Its implementation requires to determine the satisfiability of a complex constraint satisfaction problem whose size can be considerably reduced by employing a preprocessing step which crucially depends on solving Partial MaxSAT problems.**

***Index Terms*—Conditional, Knowledge Base, Nonmonotonic Reasoning, c-Inference, Partial MaxSAT**

I. Problem Domain

In knowledge representation and reasoning, the subfield of Artificial Intelligence research dealing with the formalization of, inter alia, commonsense and human-like reasoning, nonmonotonic inference relations play an important role for capturing, e.g., dynamically interrelated knowledge, defaults and uncertainty. Conditional logics constitute a main approach for nonmonotonic reasoning.

As an example for a conditional knowledge base over the signature $\Sigma_{bird} = \{b, p, f, w\}$ representing birds, penguins, flying things and winged things, let $\mathcal{R}_{bird}$ contain $r_1 = (B_1|A_1) = (f|b)$, $r_2 = (B_2|A_2) = (\overline{f}|p)$, $r_3 = (B_3|A_3) = (b|p)$, and $r_4 = (B_4|A_4) = (w|b)$. For instance, r_1 expresses *"birds usually fly"*. In this example, both consequence and antecedence of each conditional is a literal, but in general, both are arbitrary propositional formulas.

Semantical frameworks for conditionals range from quantitative probability distributions to purely qualitative approaches. Here, we make use of the semi-qualitative framework of ranking functions, also called ordinal conditional functions (OCF) [19], which assign natural numbers as degrees of implausability of possible worlds. More specifically, our work uses c-representations as a special type of such ranking functions exhibiting desirable inference properties [12, 13]. c-Representations are constructed by associating non-negative integer impacts η_i with each conditional $(B_i|A_i)$ in a knowledge base $\mathcal{R}$ and assigning to each world ω the sum of the impacts of conditionals falsified by ω, while ensuring that the resulting ranking function models $\mathcal{R}$.

Taking the set of all c-representations of a conditional knowledge base into account yields the nonmonotonic entailment relation *c-inference* [3], exhibiting notable inference properties like fulfilling the standard axioms of system P [1, 15], not suffering from the well-known drowning problem [7, 10] and fully satisfying syntax splitting [14, 18].

For realizing c-inference, solving a complex constraint satisfaction problem (CSP) is required [3, 4], involving in particular satisfaction conditions affecting all possible worlds. The general form of the constraint satisfaction problem for c-representations $CR(\mathcal{R})$ on the constraint variables $\{\eta_1, \ldots, \eta_n\}$ ranging over $\mathbb{N}_0$ is given by the constraints, for all $i \in \{1, \ldots, n\}$:

$$\eta_i \geqslant 0 \tag{1}$$

$$\eta_i > \underbrace{\min_{\omega \models A_i B_i} \sum_{\substack{j \neq i \\ \omega \models A_j \overline{B}_j}} \eta_j}_{V_{min_i}} - \underbrace{\min_{\omega \models A_i \overline{B}_i} \sum_{\substack{j \neq i \\ \omega \models A_j \overline{B}_j}} \eta_j}_{F_{min_i}} \tag{2}$$

Considering c-inference, an additional constraint for the query conditional $(B|A)$ has to be added,

$$\underbrace{\min_{\omega \models AB} \sum_{\substack{1 \leqslant i \leqslant n \\ \omega \models A_i \overline{B}_i}} \eta_i}_{V_{min_q}} \geqslant \underbrace{\min_{\omega \models A\overline{B}} \sum_{\substack{1 \leqslant i \leqslant n \\ \omega \models A_i \overline{B}_i}} \eta_i}_{F_{min_q}} \tag{3}$$

resulting in the combined CSP $CR(\mathcal{R}, A, B)$. Then, $A \mathrel{|\!\sim}^{c}_{\mathcal{R}} B$, meaning that A entails B by c-inference under knowledge base $\mathcal{R}$, holds if and only if this CSP is unsatisfiable [3, 4].

For all previously existing implementations of c-inference, the involvement of all possible worlds is a severe limiting factor, making inferences from conditional belief bases with more than about 25 conditionals and 25 signature elements, and thus 2^{25} possible worlds, practically infeasible [2, 5, 6, 8, 16, 17].

A recent approach proposes and implements a preprocessing step via solving Partial MaxSAT problems corresponding with the minimum expressions V_{min_i}, F_{min_i}, V_{min_q} and F_{min_q} in the constraints, where the satisfaction condition of each minimum expression constitutes the hard constraint of the corresponding Partial MaxSAT problem and the negations of the success conditions of the sum expressions are taken as its soft constraints [9]. One thereby achieves a significant reduction of the problem size, making feasible inferences of

conditional belief bases with up to 100 conditionals and 100 signature elements.

II. Parameters for Generation

The parameters for the generation of our benchmark problems are given by the signature size and the number of conditionals in the given knowledge base, both of which are equal in size for all submitted instances (like in the example where $|\Sigma_{bird}| = |\mathcal{R}_{bird}|$). The instances have been chosen by picking out those instances which showed the longest time to solve with the MaxSAT Solver RC2 [11]. Note that for the purpose of computing c-inference, all solutions of all corresponding Partial MaxSAT problems have to be computed during the preprocessing, meaning that for those instances with many solutions a reduction of the solving time has large practical value.

III. File Name Convention

Because the preprocessing for c-inference under a knowledge base $\mathcal{R}$ consists in solving $2|\mathcal{R}| + 2$ Partial MaxSAT problems, file names are given by *"pre-processing_c_inference_<size of signature and knowledge base>_<number of knowledge base>_<corresponding minimum expression>"*. For example, the file *pre-processing_c_inference_60_12_v5* encodes the Partial MaxSAT problem for V_{min_5} in the CSP for the knowledge base number 12 with signature size 60 and number of conditionals of 60, and *pre-processing_c_inference_70_3_fq7* encodes the Partial MaxSAT problem for F_{min_q} in the CSP for c-inference with the seventh query under knowledge base number 3 with signature size of 70 and number of conditionals of 70.

References

[1] Adams, E.: Probability and the logic of conditionals. In: Hintikka, J., Suppes, P. (eds.) Aspects of inductive logic, pp. 265–316. North-Holland, Amsterdam (1966)

[2] Beierle, C., von Berg, M., Sanin, A.: Realization of c-inference as a SAT problem. In: Keshtkar, F., Franklin, M. (eds.) Proceedings of the Thirty-Fifth International Florida Artificial Intelligence Research Society Conference (FLAIRS), Hutchinson Island, Florida, USA, May 15-18, 2022 (2022). https://doi.org/10.32473/flairs.v35i.130663

[3] Beierle, C., Eichhorn, C., Kern-Isberner, G.: Skeptical inference based on c-representations and its characterization as a constraint satisfaction problem. In: Gyssens, M., Simari, G. (eds.) Foundations of Information and Knowledge Systems - 9th International Symposium, FoIKS 2016, Linz, Austria, March 7–11, 2016. Proceedings. LNCS, vol. 9616, pp. 65–82. Springer (2016

[4] Beierle, C., Eichhorn, C., Kern-Isberner, G., Kutsch, S.: Properties and interrelationships of skeptical, weakly skeptical, and credulous inference induced by classes of minimal models. Artificial Intelligence **297**, 103489 (Aug 2021). https://doi.org/10.1016/j.artint.2021.103489

[5] Beierle, C., Eichhorn, C., Kutsch, S.: A practical comparison of qualitative inferences with preferred ranking models. KI – Künstliche Intelligenz **31**(1), 41–52 (2017). https://doi.org/10.1007/s13218-016-0453-9

[6] Beierle, C., Kutsch, S., Sauerwald, K.: Compilation of static and evolving conditional knowledge bases for computing induced nonmonotonic inference relations. Ann. Math. Artif. Intell. **87**(1-2), 5–41 (2019)

[7] Benferhat, S., Cayrol, C., Dubois, D., Lang, J., Prade, H.: Inconsistency Management and Prioritized Syntax-Based Entailment. In: Proc. IJCAI'93. vol. 1, pp. 640–647. Morgan Kaufmann Publishers, San Francisco, CA, USA (1993)

[8] von Berg, M., Sanin, A., Beierle, C.: Representing nonmonotonic inference based on c-representations as an SMT problem. In: Bouraoui, Z., Vesic, S. (eds.) Symbolic and Quantitative Approaches to Reasoning with Uncertainty - 17th European Conference, ECSQARU 2023, Arras, France, September 19-22, 2023, Proceedings. Lecture Notes in Computer Science, vol. 14294, pp. 210–223. Springer (2023)

[9] von Berg, M., Sanin, A., Beierle, C.: Scaling up nonmonotonic c-inference via partial maxsat problems. In: Meier, A., Ortiz, M. (eds.) Foundations of Information and Knowledge Systems - 13th International Symposium, FoIKS 2024, Sheffield, UK, April 8-11, 2024, Proceedings. Lecture Notes in Computer Science, vol. 14589, pp. 182–200. Springer (2024). https://doi.org/10.1007/978-3-031-56940-1_10

[10] Heyninck, J., Kern-Isberner, G., Meyer, T.A., Haldimann, J.P., Beierle, C.: Conditional syntax splitting for nonmonotonic inference operators. In: Williams, B., Chen, Y., Neville, J. (eds.) Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023, Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2023, Washington, DC, USA, February 7-14, 2023. pp. 6416–6424. AAAI Press (2023)

[11] Ignatiev, A., Morgado, A., Marques-Silva, J.: RC2: an efficient maxsat solver. J. Satisf. Boolean Model. Comput. **11**(1), 53–64 (2019)

[12] Kern-Isberner, G.: Conditionals in nonmonotonic reasoning and belief revision, LNAI, vol. 2087. Springer (2001)

[13] Kern-Isberner, G.: A thorough axiomatization of a principle of conditional preservation in belief revision. Ann. Math. Artif. Intell. **40(1-2)**, 127–164 (2004)

[14] Kern-Isberner, G., Beierle, C., Brewka, G.: Syntax splitting = relevance + independence: New postulates for nonmonotonic reasoning from conditional belief bases. In: Calvanese, D., Erdem, E., Thielscher, M. (eds.) Principles of Knowledge Representation and Reasoning: Proceedings of the 17th International Conference, KR 2020. pp. 560–571. IJCAI Organization (2020). https://doi.org/10.24963/kr.2020/56

[15] Kraus, S., Lehmann, D.J., Magidor, M.: Nonmonotonic

Reasoning, Preferential Models and Cumulative Logics. Artif. Intell. **44**(1-2), 167–207 (1990)

[16] Kutsch, S.: InfOCF-Lib: A Java library for OCF-based conditional inference. In: Beierle, C., Ragni, M., Stolzenburg, F., Thimm, M. (eds.) Proceedings of the 8th Workshop on Dynamics of Knowledge and Belief (DKB-2019) and the 7th Workshop KI & Kognition (KIK-2019) co-located with 44nd German Conference on Artificial Intelligence (KI 2019), Kassel, Germany, September 23, 2019. CEUR Workshop Proceedings, vol. 2445, pp. 47–58. CEUR-WS.org (2019)

[17] Kutsch, S., Beierle, C.: InfOCF-Web: An online tool for nonmonotonic reasoning with conditionals and ranking functions. In: Zhou, Z. (ed.) Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI 2021, Virtual Event / Montreal, Canada, 19-27 August 2021. pp. 4996–4999. ijcai.org (2021). https://doi.org/10.24963/ijcai.2021/711

[18] Parikh, R.: Beliefs, belief revision, and splitting languages. Logic, Language, and Computation **2**, 266–278 (1999)

[19] Spohn, W.: Ordinal conditional functions: a dynamic theory of epistemic states. In: Harper, W., Skyrms, B. (eds.) Causation in Decision, Belief Change, and Statistics, II, pp. 105–134. Kluwer Academic Publishers (1988)

MaxSAT Evaluation 2024: Solver and Benchmark Descriptions, volume B-2024-2 of Department of Computer Science Series of Publications B, University of Helsinki 2024.

MaxSAT encodings for Synthesizing Pareto-Optimal Interpretations for Black-Box Models*

*Benchmarks for the Main Track of the MaxSat Evaluation 2024

Aniruddha Joshi
University of California at Berkeley
Berkeley, CA, USA
aniruddhajoshi@berkeley.edu

Hazem Torfah
Chalmers University of Technology
Gothenburg, Sweden
hazemto@chalmers.se

Shetal Shah
Indian Institute of Technology Bombay
Mumbai, India
shetals@cse.iitb.ac.in

Supratik Chakraborty
Indian Institute of Technology Bombay
Mumbai, India
supratik@cse.iitb.ac.in

S. Akshay
Indian Institute of Technology Bombay
Mumbai, India
akshayss@cse.iitb.ac.in

Sanjit A. Seshia
University of California at Berkeley
Berkeley, CA, USA
sseshia@berkeley.edu

***Abstract*—In Pareto-optimal interpretation synthesis, we start with a black-box ML model and automatically synthesize a set of interpretations such that each interpretation is Pareto-optimal with respect to suitably defined measures. The user can choose a finite class of syntactic templates from which an interpretation must be synthesized, and also suitable quantitative measures for correctness and explainability of every interpretation in this class. The resulting multi-objective optimization problem can be solved via a reduction to quantitative constraint solving, such as weighted maximum satisfiability. For purposes of generating the benchmarks, we used hand-crafted decision diagrams as (substitutes for black-box) ML models. Our benchmarks include encodings of the Pareto-optimal interpretation synthesis problem for these decision diagram (treated as black-box models), where an upper bound on the size of the interpretation is varied.**

***Index Terms*—Explainability, Pareto-optimal synthesis, Multi-objective optimization**

I. Introduction

To synthesize an interpretation of a black-box ML model, one often needs to strike a balance between the correctness or accuracy of the interpretation (measured in terms of fidelity, misclassification rate of predictions, etc.) and its explainability or understandability (approximated by the size/depth of decision tree/list/diagram, number and nature of predicates used, etc.) [1], [2]. In [4], we proposed and implemented an algorithm for synthesizing a set of Pareto-optimal interpretations from a combinatorially large but finite class of possible interpretations. We encoded the synthesis problem as a weighted maximum satisfiability (MaxSAT) problem. This MaxSAT encoding is a conjunction of four formulae $\phi_\epsilon \wedge \phi_S \wedge \phi_{\Delta_C} \wedge \phi_{\Delta_\epsilon}$. The formula ϕ_ϵ encodes the syntactic restrictions on the interpretations, and ϕ_S encodes the semantic constraints. Semantic constraints establish a relation between samples from the ML model and an interpretation satisfying ϕ_ϵ. Formulae ϕ_{Δ_C} and ϕ_{Δ_ϵ} encode correctness and explainability constraints, respectively. These correspond to the structure of an interpretation including the choice of predicates and labels. In particular, interpretations that have high correctness and explainability measures are desirable. The algorithm solved a series of these encodings, where each encoding constrained the region of the interpretation space by enforcing upper and lower bounds on the explainability measure.

The weighted maximum satisfiability encodings used in [4] were submitted to [5]. However, we realized later that some of these benchmarks turned out to be easy instances of weighted MaxSAT solving. In the current submission, we have therefore focused on generating harder problem instances. Specifically, we have followed the same encoding principle as used in the previous year, but have used specially constructed decision diagrams as substitutes for black box ML models, with the objective of generating a good mix of easy and hard weighted MaxSAT instances.

II. Benchmark Description

We now describe the benchmarks; specifically the custom-designed decision diagrams that have been used as substitutes for black box ML models. The weighted MaxSAT encodings for these interpretations are generated using the methodology mentioned in [3], [4].

We use four different classes of synthetically generated decision diagrams. Each internal node (decision node) of the diagram represents a feature of the model, and the leaves represent the labels or output of the decision diagram. In our experience, a combination of high arity (the number of outgoing edges from an internal node) and high number of

This work was partially supported by NSF grants 1545126 (VeHICaL), 1646208 and 1837132, by the DARPA contracts FA8750-18-C-0101 (AA) and FA8750-20-C-0156 (SDCPS), by Berkeley Deep Drive, by Toyota under the iCyPhy center, and by grant TIH-IoT/06-2022/IC/NSF/SL/NIUC-2022-05/001 from TiH-IoT at IIT Bombay for NSF-DST jointly funded research projecs.

internal nodes at every depth (minimum path length from the root) seems to result in encodings which are not easy to solve. To obtain a good mix of easy and hard to solve instances, we chose the following 4 classes of decision diagrams:

1) **Complete N-ary Trees:** This class of black box models consists of complete n-ary trees. These trees are parameterized by arity N and depth D. The trees in this class have depth and arity ranging from 2 to 5. All internal nodes of a tree in this class have the same arity.
2) **Trees with Dichotomous Branching:** This class represents a tree with two long paths of equal length. The trees in this class are parameterized by length L of each path. The class contains trees where L varies from 2 to 4. Every tree contains $2L+1$ internal nodes and $2L+2$ labels.
3) **Single Path Trees:** This class represents a tree with one long path and labels attached to each node of the path. This class is parameterized with the length L of the path. It contains $L+1$ labels and L features, i.e., internal nodes. The class contains trees where L varies from 2 to 5.
4) **Random Trees:** This class consists of randomly generated decision diagrams, where the number of labels are uniformly randomly chosen from 2 to 12. Each diagram has an associated maximum arity max_arity which is uniformly randomly chosen between 2 to 7. All the internal nodes of the diagram have an arity which is uniformly randomly chosen from 2 to max_arity. Each diagram has an associated max_size which is chosen uniformly randomly from 2 to 10, the number of decision nodes in the diagram are restricted to be less than the max_size.

For each black box model, we generate the MaxSAT encodings for the Pareto-optimal interpretation synthesis problem for different sizes of the decision diagrams.

REFERENCES

[1] A. Adadi and M. Berrada. "Peeking inside the black-box: A survey on Explainable Artificial Intelligence (XAI)". IEEE Access, 2018.

[2] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, Franco, F. Giannotti, and D. Pedreschi. "A Survey of Methods for Explaining Black Box Models". ACM Comput. Surv., 2018.

[3] H. Torfah, S. Shah, S. Chakraborty, S. Akshay and S. A. Seshia, "Synthesizing Pareto-Optimal Interpretations for Black-Box Models", CoRR arXiv, abs/2108.07307, http://arxiv.org/abs/2108.07307, 2021.

[4] H. Torfah, S. Shah, S. Chakraborty, S. Akshay and S. A. Seshia, "Synthesizing Pareto-Optimal Interpretations for Black-Box Models". Formal Methods in Computer Aided Design, FMCAD 2021, New Haven, CT, USA, October 19-22, https://doi.org/10.34727/2021/isbn. 978-3-85448-046-4_24, 2021.

[5] MaxSAT Evaluation 2023 lhttps://maxsat-evaluations.github.io/2023/

Incremental MaxSAT benchmarks from dynamic discretizations of train scheduling problems

Bjørnar Luteberget
SINTEF Digital AS
Oslo, Norway
bjornar.luteberget@sintef.no

SAT and MaxSAT solvers are good choices for solving scheduling problems when there is a natural and coarse discretization of time, such as in academic timetabling [1]. In the railway domain, online/real-time scheduling applications have typically not been a good match for time discretization approaches (see [2]–[4]), because there is often no reasonable choice of coarseness: when the discretization is too fine, the problem instance becomes too large to solve in a reasonable amount of time, and when the discretization is too coarse, the approximate solutions may prove sub-optimal or even infeasible in the field.

Dynamic Discretization Discovery (DDD), recently introduced by [5], is a technique used to convert scheduling problems over continuous time into discrete variables in a way that mitigates these size and approximation issues. By solving a sequence of smaller problems, the discretization is built incrementally in a way that ensures convergence to an optimal solution of the complete continuous formulation.

In [6], the DDD technique was adapted to a the train dispatching problem, which is a type of railway scheduling problem that is solved in an online setting where some trains have become delayed and are no longer able to follow their original timetables. The objective of the train dispatching problem in [6] is to reschedule the trains to minimize the sum of delays. The measure of the delay may either be the continuous numerical value of the delay, or a step-wise function that defines constant costs for exceeding some defined delay thresholds. For example, if a train is delayed by 3 minutes it could incur a cost of 1, if it is more than 10 minutes delayed, a cost of 2, and so on.

The paper [6] compares two approaches for solving the train dispatching problem: the commercial mixed-integer programming solver Gurobi using continuous variables, and the dynamic discretization (DDD) solved using the RC2 MaxSAT algorithm [7]. On step-wise objective functions, the MaxSAT approach had better performance.

The IPAMIR Train Scheduling benchmark submitted to the MaxSAT Evaluation 2024 is a command-line application that solves the train scheduling problems from [6] with a step-wise objective function. The program uses incremental calls to a MaxSAT solver through the IPAMIR interface. It calls IPAMIR to add hard clauses and to set literal weights (but only on literals that previously did not have an assigned weight, i.e., the weights are non-decreasing). The benchmarking program uses the *linear rounded* objective defined in [6], i.e. the cost is $c(d) = \lfloor d/Q \rfloor$, for each train's delay d, with $Q = 180$ sec.

The input files provided with the benchmarking program are 72 problem instances constructed from real-world railway data from two single-track railroad networks, named Line A and Line B, extracted from the real-time train information system of the Norwegian state railways. Line A is 124 km long, includes 30 stations and an average of 20 trains per problem instance. Line B is 115 km, includes 20 stations and an average of 11 trains per problem instance. There are 24 original instances from the information system, named in the style `origA01.txt`, where `A` is the railway line and `01` is the instance number. In addition, two types of modifications were made to create harder problem instances: instances named `track*.txt` have increased traveling time between stations, and instances named `station*.txt` have increased station dwelling times. See [6] for more details.

References

[1] R. Asín Achá and R. Nieuwenhuis, "Curriculum-based course timetabling with SAT and MaxSAT," *Annals of Operations Research*, vol. 218, pp. 71–91, 2014.

[2] C. Mannino and A. Mascis, "Optimal real-time traffic control in metro stations," *Operations Research*, vol. 57, no. 4, pp. 1026–1039, 2009.

[3] S. Harrod, "Modeling network transition constraints with hypergraphs," *Transportation Science*, vol. 45, no. 1, pp. 81–97, 2011.

[4] N. L. Boland and M. W. Savelsbergh, "Perspectives on integer programming for time-dependent models," *Top*, vol. 27, no. 2, pp. 147–173, 2019.

[5] N. Boland, M. Hewitt, L. Marshall, and M. Savelsbergh, "The continuous-time service network design problem," *Operations research*, vol. 65, no. 5, pp. 1303–1321, 2017.

[6] A. L. Croella, B. Luteberget, C. Mannino, and P. Ventura, "A MaxSAT approach for solving a new Dynamic Discretization Discovery model for train rescheduling problems," *Computers & Operations Research*, p. 106 679, 2024.

[7] A. Ignatiev, A. Morgado, and J. Marques-Silva, "RC2: an efficient MaxSAT solver," *J. Satisf. Boolean Model. Comput.*, vol. 11, no. 1, pp. 53–64, 2019. DOI: 10.3233/SAT190116. [Online]. Available: https://doi.org/10.3233/SAT190116.

Learning Balanced Rules via MaxSAT: Benchmark Description

Antônio Carlos Souza Ferreira Júnior
Federal Institute of Ceará (IFCE)
Brazil

Thiago Alves Rocha
Federal Institute of Ceará (IFCE)
Brazil
thiago.alves@ifce.edu.br

***Abstract*—This paper presents benchmarks for a MaxSAT-based model, IMLIB, designed to learn balanced classification rules. IMLIB emphasizes rule size balance for enhanced interpretability. The benchmarks consist of 314 instances generated from datasets of the UCI repository.**

***Index Terms*—MaxSAT, Benchmarks, Rule Learning**

I. Introduction

The success of Machine Learning (ML) has led to advancements in diverse applications. A significant challenge is the lack of explainability in prediction models, impacting their reliability in critical areas such as finance, education, and medicine. To address this, some works focus on enhancing explainability in predictions by creating rule-based classifiers, which are particularly effective for providing interpretations to end users [1], [2]. Achieving precise predictions with high interpretability is computationally hard, and several studies strive to balance accuracy and interpretability in their models.

We designed a rule-based classifier called IMLIB[1] to learn rules with limited size [3]. This approach allows us to formulate the learning problem as a MaxSAT problem, generating smaller and more balanced rules. We argue that a rule-based classifier, where all rules are small, seems more interpretable than one with fewer rules, where some of them are large. Our method also adopts an incremental technique as described in [2], enabling efficient training on large datasets.

II. MaxSAT Approach for Learning Balanced Rules

We are given a dataset $(\mathbf{X}, \mathbf{y})$ of n samples, each sample with m binary features. Given also k and l, we learn a set of rules $\mathbf{R}$ represented by a DNF with k terms, each with at most l literals. The constraints of the MaxSAT instances are defined over variables: $u^j_{o,d}$, $p_{o,d}$, $u^*_{o,d}$, $e_{o,d,i}$ and $z_{o,i}$ such that $o \in \{1, ..., k\}$, $d \in \{1, ..., l\}$, $j \in \{1, ..., m\}$ and $i \in \{1, ..., n\}$. If the valuation of $u^j_{o,d}$ is true, it means that the jth feature will be the dth feature of the rule R_o. Furthermore, if $p_{o,d}$ is true, it means that the dth feature of the rule R_o is positive. Otherwise, it is negative. If $u^*_{o,d}$ is true, it means that the dth feature is skipped in the rule R_o. In this case, we ignore $p_{o,d}$. If $e_{o,d,i}$ is true, then the dth feature of rule R_o contributes to the correct classification of the ith sample. If $z_{o,i}$ is true, then the rule R_o contributes to the correct classification of the ith sample.

Given also a penalization cost λ, the set of rules $\mathbf{R}$ has a trade-off between the number of literals $|\mathbf{R}|$ in $\mathbf{R}$ and the classification error, penalizing classification errors with λ. More formally, our method solves the optimization problem $\min_{\mathbf{R}}\{|\mathbf{R}| + \lambda|\mathcal{E}_R| \mid \mathcal{E}_R = \{\mathbf{X}_i \mid \mathbf{R}(\mathbf{X}_i) \neq y_i\}\}$ such that $\mathbf{R}$ has k rules, each with at most l literals. The constraints of the MaxSAT instances are defined bellow. The weight of a constraint is defined by $W(\cdot)$. Constraints that are not in clause form are converted to clauses, with the weight of each constraint being applied to all corresponding clauses.

Constraints that guarantee that exactly only one $u^j_{o,d}$ is true for the dth feature of the rule R_o:

$$A = \bigvee_{j \in \{1,...,m,*\}} u^j_{o,d}, W(A) = \infty \tag{1}$$

$$B = \neg u^j_{o,d} \vee \neg u^{j'}_{o,d}, W(B) = \infty \tag{2}$$

In constraint B, we have $j, j' \in \{1, ..., m, *\}$ such that $j \neq j'$. Constraints representing that the model will try to insert as few features as possible in $\mathbf{R}$:

$$C_1 = \neg u^j_{o,d}, W(C_1) = 1 \tag{3}$$

$$C_2 = u^*_{o,d}, W(C_2) = 1 \tag{4}$$

In constraint C_1, we have $j \in \{1, ..., m\}$. Clauses that guarantees that each rule has at least one feature:

$$D = \bigvee_{d \in \{1,...,l\}} \neg u^*_{o,d}, W(D) = \infty \tag{5}$$

Constraints ensuring that $e_{o,d,i}$ is true if the jth feature in the oth rule contributes to correctly classify sample $\mathbf{X}_i$:

$$E = u^j_{o,d} \rightarrow (p_{o,d} \leftrightarrow s^j_{o,d,i}), W(E) = \infty \tag{6}$$

Constraints guaranteeing that the classification of a specific rule will not be interfered by skipped features in the rule:

$$F = u^*_{o,d} \rightarrow e_{o,d,i}, W(F) = \infty \tag{7}$$

Constraints indicating that the model assigns true to $z_{o,i}$ if all the features of rule R_o support the correct classification of sample $\mathbf{X}_i$:

$$G = z_{o,i} \leftrightarrow \bigwedge_{d \in \{1,...,l\}} e_{o,d,i}, W(G) = \infty \tag{8}$$

[1]Source code of IMLIB can be found at the link: https://github.com/cacajr/decision_set_models

Clauses designed to generate a set of rules **R** that correctly classify as many samples as possible:

$$H = \bigwedge_{i \in \mathcal{E}^+} \bigvee_{o \in \{1,\ldots,k\}} z_{o,i}, W(H) = \lambda \tag{9}$$

$$I = \bigwedge_{i \in \mathcal{E}^-} \bigwedge_{o \in \{1,\ldots,k\}} \neg z_{o,i}, W(I) = \lambda \tag{10}$$

In the above constraints, $\mathcal{E}^+$ and $\mathcal{E}^-$ represent the indexes of positive and negative samples, respectively. More formally, $\mathcal{E}^- = \{i \mid y_i = 0, 1 \leq i \leq n\}$ and $\mathcal{E}^+ = \{i \mid y_i = 1, 1 \leq i \leq n\}$. Moreover, the cost λ is a parameter that handles the trade-off between rule-sparsity and prediction accuracy.

IMLIB can also partition the set of samples **X**. In this case, all constraints described above are applied in the first partition. Starting from the second partition, the constraints in (3) and (4) are replaced by the following constraints:

$$C' = \begin{cases} u^j_{o,d}, \text{ if } u^j_{o,d} \text{ is true in the previous} \\ \text{partition;} \\ \neg u^j_{o,d}, \text{ otherwise.} \end{cases} , W(C') = 1 \tag{11}$$

III. Benchmark Instances

The **Weighted Partial MaxSAT** instances are generated from 8 datasets of the UCI repository. More precisely, the datasets are binarized, ensuring compatibility with the MaxSAT formulation. The number of samples in the selected datasets ranges from 53 to 1286. For each dataset, we generate a MaxSAT instance to train IMLIB with all samples in the dataset. Additionally, we create MaxSAT instances using dataset partitioning and the incremental technique. We considered 4 parameters to generate the instances: the number of rules k, the penalization cost λ, the maximum number of literals per rule l, and the partition size s. Given a partition size s, we define the number of partitions as $p = \lceil \frac{n}{s} \rceil$, where n is the number of samples in the training dataset. Then, we create $n\%p$ (the remainder when dividing n by p) partitions of size $\lfloor \frac{n}{p} \rfloor + 1$ and the rest of size $\lfloor \frac{n}{p} \rfloor$.

To create the MaxSAT instances using partitioning, we employed a validation process for each dataset to evaluate performance in terms of accuracy on test data. Based on this evaluation, we selected the best parameter values for k, l, λ, and s. We considered the following parameter values: $k \in \{1, 2, 3\}$, $\lambda \in \{5, 10\}$, and $s \in \{8, 16\}$. With respect to l, we chose the best value obtained in comparison with other models [2], as described in [3]. For the MaxSAT instances to train IMLIB with all samples in the dataset, we used the same parameter values as in the incremental case. Therefore, each MaxSAT instance file has the following naming convention:

dataset_p_idPartition_#samples_k_l_λ.wcnf

This means each instance file starts with the dataset name, followed by the number of partitions p, the partition ID (a value between 1 and p), the number of samples in the current partition, and the parameter values k, l, λ, respectively. For example, the file named:

iris_9_2_15_1_4_5.wcnf

contains a MaxSAT instance in WCNF format that encodes the problem of training IMLIB on the 2nd of 9 partitions of the iris dataset. This partition has 15 samples and IMLIB is trained with parameter values $k = 1$, $l = 4$, and $\lambda = 5$.

References

[1] A. Ignatiev, J. Marques-Silva, N. Narodytska, and P. J. Stuckey, "Reasoning-based learning of interpretable ML models," in *30th IJCAI*, 2021, pp. 4458–4465, https://doi.org/10.24963/ijcai.2021/608.

[2] B. Ghosh, D. Malioutov, and K. S. Meel, "Efficient learning of interpretable classification rules," *Journal of Artificial Intelligence Research*, vol. 74, pp. 1823–1863, 2022, https://doi.org/10.1613/jair.1.13482.

[3] A. C. S. Ferreira Júnior and T. A. Rocha, "An incremental MaxSAT-based model to learn interpretable and balanced classification rules," in *Intelligent Systems: 12th BRACIS*, 2023, p. 227–242, https://doi.org/10.48550/arXiv.2403.16418.

MaxSAT 2024 Benchmarks: Minimizing Pentagons in the Plane

Bernardo Subercaseaux
Carnegie Mellon University
bsuberca@andrew.cmu.edu

John Mackey
Carnegie Mellon University
jmackey@andrew.cmu.edu

Marijn J.H. Heule
Carnegie Mellon University
mheule@andrew.cmu.edu

Ruben Martins
Carnegie Mellon University
rubenm@andrew.cmu.edu

***Abstract*—What is $\mu_5(n)$, the minimum number of convex pentagons induced by n points in the plane? A surprisingly simple upper bound follows from the construction presented in our paper [1]: $\mu_5(n) \leq \binom{\lfloor n/2 \rfloor}{5} + \binom{\lceil n/2 \rceil}{5}$, and we conjecture it is optimal. Using MaxSAT, we confirm that $\mu_5(n)$ matches the conjectured values for $n \leq 16$. This benchmark description describes the MaxSAT encoding and benchmarks submitted to the MaxSAT Evaluation 2024.**

I. The Pentagon Minimization Problem

A set of points $S = \{p_1, \ldots, p_n\}$ in the plane (i.e., $p_i = (x_i, y_i) \in \mathbb{R}^2$) is *in general position* when no subset of three points of S belong to a common straight line. Given an integer value n, we aim to find a set S of n points in general position that minimizes the number of convex 5-gons with vertices in S. In order to address this problem computationally one needs to shift from the continuous space $\mathbb{R}^2$ to a discrete and finitely-representable abstraction. The crucial observation to achieve this is that the number of convex pentagons on a set of points does not depend on their exact positions (e.g., it is invariant under scaling or rotations) but rather on the relationship between them.

This is illustrated in Figures 1a and 1b; each of them depicts a pair of pentagons that differ with respect to the exact position of their vertices but are *equivalent* in terms of the relationships between vertices, in a sense that we make precise next. As it is standard in the area, we abstract a set of points S according to the relative orientation of each of its subsets of three points [2]–[5]. We assume without loss of generality that the points are labeled from left to right, that is, for every triple $p_a, p_b, p_c \in S$, with $a < b < c$, we assume $x_a < x_b < x_c$ [2]. Then, for every triple $a < b < c$ we define its *signotope* $\sigma(a, b, c)$ as true if p_a, p_b, p_c appear in counterclockwise order, and false otherwise. Formally,

$$\sigma(a,b,c) = \begin{cases} \mathsf{true} & \text{if } (y_c - y_a)(x_b - x_a) > \\ & \quad (x_c - x_a)(y_b - y_a), \\ \mathsf{false} & \text{otherwise.} \end{cases}$$

Importantly, not every combination of signotopes is *consistent* with the left-to-right labeling we assume. For a minimal example, consider points p_a, p_b, p_c, p_d, with $a < b < c < d$, such that p_a, p_b, p_c appear in clockwise order, and p_b, p_c, p_d also appear in clockwise order. Then, necessarily, p_a, p_c, p_d must appear in clockwise order as well, which translates to the implication $\neg\sigma(a,b,c) \wedge \neg\sigma(b,c,d) \implies \neg\sigma(a,c,d)$. More in general, the following *signotope axioms* apply to any set of points in general position, provided that the points are sorted with respect to their x-coordinates [2], [6]:

$$\big(\ \sigma(a,b,c) \vee \neg\sigma(a,b,d) \vee \ \sigma(a,c,d)\big) \ \wedge$$
$$\big(\neg\sigma(a,b,c) \vee \ \sigma(a,b,d) \vee \neg\sigma(a,c,d)\big) \tag{1}$$
$$\big(\ \sigma(a,b,c) \vee \neg\sigma(a,c,d) \vee \ \sigma(b,c,d)\big) \ \wedge$$
$$\big(\neg\sigma(a,b,c) \vee \ \sigma(a,c,d) \vee \neg\sigma(b,c,d)\big) \tag{2}$$
$$\big(\ \sigma(a,b,c) \vee \neg\sigma(a,b,d) \vee \ \sigma(b,c,d)\big) \ \wedge$$
$$\big(\neg\sigma(a,b,c) \vee \ \sigma(a,b,d) \vee \neg\sigma(b,c,d)\big) \tag{3}$$
$$\big(\ \sigma(a,b,d) \vee \neg\sigma(a,c,d) \vee \ \sigma(b,c,d)\big) \ \wedge$$
$$\big(\neg\sigma(a,b,d) \vee \ \sigma(a,c,d) \vee \neg\sigma(b,c,d)\big) \tag{4}$$

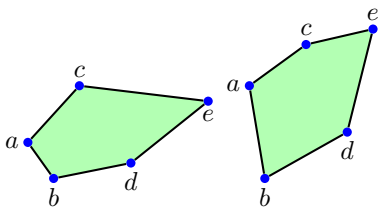
II. MaxSAT Encoding

We will use the signotopes $\sigma(a,b,c)$ directly as propositional variables in our encoding. As a first step, we directly add the $O(n^4)$ axiom clauses of Equations (1)-(4). Then, in order to minimize the number of induced convex 5-gons, we use an idea of Szekeres and Peters [2]. Szekeres and Peters identified the four cases that form a convex 5-gon, depending on where the three middle points b, c, and d are located with respect to the line through the leftmost point a and the rightmost point e:

Case I: $\sigma(a,b,c) = \ \sigma(b,c,d) = \ \sigma(c,d,e)$
Case II: $\sigma(a,b,c) = \ \sigma(b,c,e) = \neg\sigma(a,d,e)$
Case III: $\sigma(a,b,d) = \ \sigma(b,d,e) = \neg\sigma(a,c,e)$
Case IV: $\sigma(a,b,e) = \neg\sigma(a,c,d) = \neg\sigma(c,d,e)$.

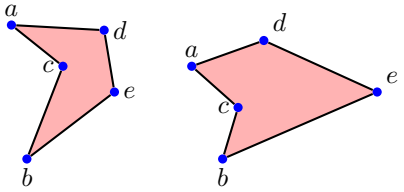
The four cases are illustrated in Figure 2, showcasing that each case has two possible orientations depending on the value of the signotopes that the case asserts to be equal.

To build a MaxSAT encoding for this problem, we first introduce *relaxation* variables $r(a,b,c,d,e)$ to denote whether the 5-gon with vertices a, b, c, d, and e is convex and thus must be avoided. We then add the following clauses based on the disjoint cases **I**-**IV**, for every sorted tuple of five points (a,b,c,d,e):



$\sigma(a,b,c) = \mathsf{true}$ $\sigma(a,d,e) = \mathsf{true}$
$\sigma(a,b,d) = \mathsf{true}$ $\sigma(b,c,d) = \mathsf{false}$
$\sigma(a,b,e) = \mathsf{true}$ $\sigma(b,c,e) = \mathsf{false}$
$\sigma(a,c,d) = \mathsf{false}$ $\sigma(b,d,e) = \mathsf{true}$
$\sigma(a,c,e) = \mathsf{false}$ $\sigma(c,d,e) = \mathsf{true}$

(a) A pair of convex pentagons that are equivalent with respect to their signotopes.



$\sigma(a,b,c) = \mathsf{true}$ $\sigma(a,d,e) = \mathsf{false}$
$\sigma(a,b,d) = \mathsf{true}$ $\sigma(b,c,d) = \mathsf{false}$
$\sigma(a,b,e) = \mathsf{true}$ $\sigma(b,c,e) = \mathsf{false}$
$\sigma(a,c,d) = \mathsf{true}$ $\sigma(b,d,e) = \mathsf{false}$
$\sigma(a,c,e) = \mathsf{false}$ $\sigma(c,d,e) = \mathsf{false}$

(b) A pair of non-convex pentagons that are equivalent with respect to their signotopes.

Fig. 1: Illustration of the signotope abstraction.

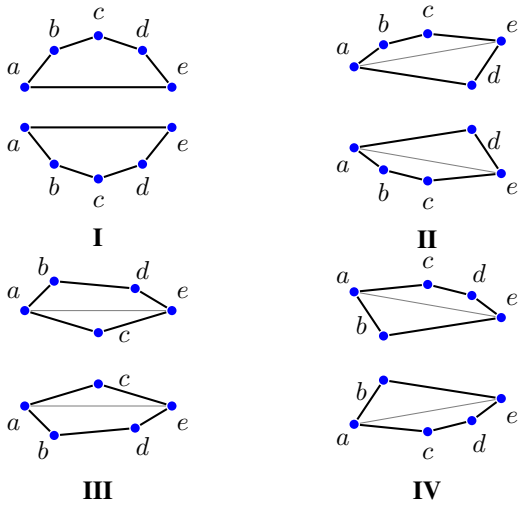


Fig. 2: The convex 5-gon cases based on the position of b, c, d w.r.t. the line $\overline{ae}$.

$$\sigma(a,b,c) \vee \sigma(b,c,d) \vee \sigma(c,d,e) \vee r(a,b,c,d,e) \quad (5)$$
$$\neg\sigma(a,b,c) \vee \neg\sigma(b,c,d) \vee \neg\sigma(c,d,e) \vee r(a,b,c,d,e) \quad (6)$$
$$\sigma(a,b,c) \vee \sigma(b,c,e) \vee \neg\sigma(a,d,e) \vee r(a,b,c,d,e) \quad (7)$$
$$\neg\sigma(a,b,c) \vee \neg\sigma(b,c,e) \vee \sigma(a,d,e) \vee r(a,b,c,d,e) \quad (8)$$
$$\sigma(a,b,d) \vee \sigma(b,d,e) \vee \neg\sigma(a,c,e) \vee r(a,b,c,d,e) \quad (9)$$
$$\neg\sigma(a,b,d) \vee \neg\sigma(b,d,e) \vee \sigma(a,c,e) \vee r(a,b,c,d,e) \quad (10)$$
$$\sigma(a,b,e) \vee \neg\sigma(a,c,d) \vee \neg\sigma(c,d,e) \vee r(a,b,c,d,e) \quad (11)$$
$$\neg\sigma(a,b,e) \vee \sigma(a,c,d) \vee \sigma(c,d,e) \vee r(a,b,c,d,e). \quad (12)$$

The axiom clauses (1)-(4) in Section I and the modified clauses (5)-(12) are defined as being hard. Finally we introduce the following soft clauses:

$$\bigwedge_{(a,b,c,d,e)\in S} \big(\neg r(a,b,c,d,e)\big). \quad (13)$$

The MaxSAT formulas for $\mu_5(n)$ are modest in size for small n, with $\mu_5(15)$ featuring about 3 500 variables, 35 000 hard clauses, and 3 000 soft clauses. Despite being small in size, this problem is particularly challenging for MaxSAT solvers.

A. Symmetry Breaking

Adding symmetry-breaking predicates that remove equivalent solutions can prune the search space and improve the performance of SAT solvers [7], [8] and is beneficial as well for MaxSAT solvers. For $\mu_5(n)$, we can break some symmetries by adding hard unit clauses $\sigma(1,b,c)$ with $b < c$ so that only solutions where points appear in counterclockwise order with respect to p_1 are obtained.

B. Cube-and-conquer

We explore a strategy inspired by the cube-and-conquer approach to parallelizing SAT formulas [9]. The key idea behind cube-and-conquer is to split a formula into 2^n disjoint formulas by carefully choosing n variables and fixing their truth values to all possible 2^n combinations. For instance, consider Boolean variables x_1 and x_2 that belong to a formula φ. This formula can be split into 4 disjoint formulas with the following construction $\varphi_1 = \varphi \cup (x_1 \wedge x_2)$, $\varphi_2 = \varphi \cup (\neg x_1 \wedge x_2)$, $\varphi_3 = \varphi \cup (x_1 \wedge \neg x_2)$, and $\varphi_4 = \varphi \cup (\neg x_1 \wedge \neg x_2)$. The intuition behind this idea is that it is easier to solve φ_i than φ and that this approach can be used to create many disjoint formulas and enable massive parallelism. In our evaluation, we selected the variables $\sigma(3,4,5)$, $\sigma(5,6,7)$, $\ldots$, $\sigma(n-2,n-1,n)$ (3 for $\mu_5(9)$, 4 for $\mu_5(11)$, 5 for $\mu_5(13)$, and 6 for $\mu_5(15)$), because we experimentally observed that these variables split

the search into balanced subspaces. Note that an optimal solution for φ corresponds to the best optimal solution for *all* φ_i.

C. MaxSAT Evaluation 2024

For the MaxSAT Evaluation 2024, we submitted 64 instances that correspond to MaxSAT formulas for the cube-and-conquer approach for $\mu_5(15)$ and 1 instance that corresponds to the original formula without the cube-and-conquer approach. In our experimental evaluation, MaxCDCL [10] without ILP preprocessing was the best solver for this problem. More details can be found in the paper [1].

ACKNOWLEDGMENT

This work is partially supported by the U.S. National Science Foundation (NSF) under grant CCF-2108521.

REFERENCES

[1] B. Subercaseaux, J. Mackey, M. J. Heule, and R. Martins, "Automated mathematical discovery and verification: Minimizing pentagons in the plane," in *International Conference on Intelligent Computer Mathematics*. Springer, 2024, pp. 21–41.

[2] G. Szekeres and L. Peters, "Computer solution to the 17-point Erdős-Szekeres problem," *The ANZIAM Journal*, vol. 48, no. 2, p. 151–164, 2006.

[3] D. E. Knuth, *Axioms and Hulls*, ser. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 1992, p. 1–98.

[4] M. Scheucher, "Two disjoint 5-holes in point sets," *Computational Geometry*, vol. 91, Dec. 2020.

[5] ——, "A sat attack on Erdős-Szekeres numbers in r^d and the empty hexagon theorem," *Computing in Geometry and Topology*, vol. 2, no. 1, pp. 2:1–2:13, Mar. 2023.

[6] S. Felsner and H. Weil, "Sweeps, arrangements and signotopes," *Discrete Applied Mathematics*, vol. 109, no. 1, p. 67–94, Apr. 2001.

[7] J. Devriendt, B. Bogaerts, M. Bruynooghe, and M. Denecker, "Improved static symmetry breaking for SAT," in *Theory and Applications of Satisfiability Testing - SAT 2016 - 19th International Conference*, ser. Lecture Notes in Computer Science, N. Creignou and D. L. Berre, Eds., vol. 9710. Springer, 2016, pp. 104–122.

[8] H. Metin, S. Baarir, M. Colange, and F. Kordon, "Cdclsym: Introducing effective symmetry breaking in SAT solving," in *Tools and Algorithms for the Construction and Analysis of Systems - 24th International Conference, TACAS 2018, Held as Part of ETAPS, Proceedings, Part I*, ser. Lecture Notes in Computer Science, D. Beyer and M. Huisman, Eds., vol. 10805. Springer, 2018, pp. 99–114.

[9] M. J. H. Heule, O. Kullmann, and A. Biere, "Cube-and-conquer for satisfiability," in *Handbook of Parallel Constraint Reasoning*, Y. Hamadi and L. Sais, Eds. Springer, 2018, pp. 31–59.

[10] C. Li, Z. Xu, J. Coll, F. Manyà, D. Habet, and K. He, "Combining clause learning and branch and bound for maxsat," in *27th International Conference on Principles and Practice of Constraint Programming*, ser. LIPIcs, L. D. Michel, Ed., vol. 210. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021, pp. 38:1–38:18.

Benchmark Submission to the Main Track of MaxSAT Evaluation 2024

Wenxi Wang
The University of Texas at Austin
wenxiw@utexas.edu

Yang Hu
The University of Texas at Austin
huyang@utexas.edu

I. Introduction

We are researchers from The University of Texas at Austin, working on the intersection of automated reasoning and software security. Our recent work [1] explores the application of MaxSAT combined with Graph Neural Networks to fix Privilege Escalation (PE) [2] issues in cloud access control misconfigurations, which has been accepted by the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE'23). We notice that our MaxSAT encoding in this work can be reused to synthesize benchmarks for evaluating MaxSAT solvers. Given this, we are presenting this benchmark submission, comprising 127 MaxSAT problem instances. In the following sections, we introduce the problem domain and our benchmark synthesis methodology.

II. Problem Domain

Identity and Access Management (IAM) [3] is an access control service employed within cloud platforms. Customers must configure IAM to establish secure access control rules for their cloud organizations. However, IAM misconfigurations can be exploited to conduct PE attacks [2], resulting in significant financial losses. Consequently, addressing these PEs is crucial for improving security assurance for cloud customers.

In our ASE'23 paper [1], we propose a novel IAM Privilege Escalation Repair Engine called `IAMPERE` that efficiently generates an approximately minimal patch for repairing a broad range of IAM PEs. To achieve this, we first formulate the IAM PE repair problem into a MaxSAT problem. To improve scalability, we first apply a carefully designed Graph Neural Network model to generate an intermediate patch that is relatively small, but not necessarily minimal. We then use a MaxSAT solver to search for a minimum repair within the space defined by the intermediate patch, as the final approximately minimum patch. Experimental results show that `IAMPERE` fixes a significantly larger number of IAM misconfigurations with markedly smaller patch sizes compared to the baseline called `IAM-Deescalate` [4]. More details about `IAMPERE` can be found in our paper[1] and slides [2].

III. Benchmark Synthesis Methodology

We initially employ the `IAMVulGen` tool [5] with default setting to produce 200 IAM misconfigurations with PEs. For each of these misconfigurations, we attempt to create a MaxSAT problem instance for fixing its PE. To avoid generating exceedingly intricate instances, we have set a 6GB memory limit and a 500-second time limit for synthesizing each problem instance. As a result, we successfully generated 127 MaxSAT problem instances.

Each instance adheres to the latest `wcnf` format[3], exclusively incorporating unweighted soft clauses (i.e., the weight of each soft clause is set to one) and hard clauses. Each instance is archived within a distinct gzipped tar file, named `cloud-access-control-repair_[ID].tar.gz`, where `[ID]` corresponds to the ID tied to its corresponding IAM misconfiguration.

References

[1] Y. Hu, W. Wang, S. Khurshid, K. L. McMillan, and M. Tiwari, "Fixing privilege escalations in cloud access control with maxsat and graph neural networks," in *38th IEEE/ACM International Conference on Automated Software Engineering (ASE 2023)*. IEEE, 2023.

[2] S. Gietzen, "Aws iam privilege escalation – methods and mitigation," https://rhinosecuritylabs.com/aws/aws-privilege-escalation-methods-mitigation/, 2018.

[3] AWS, "Aws identity and access management (iam)," https://aws.amazon.com/iam/, 2023.

[4] J. Chen, "Iam-deescalate: An open source tool to help users reduce the risk of privilege escalation," https://unit42.paloaltonetworks.com/iam-deescalate/, 2022.

[5] Y. Hu, W. Wang, S. Khurshid, and M. Tiwari, "Interactive greybox penetration testing for cloud access control using iam modeling and deep reinforcement learning," *arXiv preprint arXiv:2304.14540*, 2023.

[1] https://huyang-utspark.github.io/papers/ase23.pdf

[2] https://huyang-utspark.github.io/slides/ase23-slides.pdf

[3] https://maxsat-evaluations.github.io/2023/rules.html#input

Solver Index

Benchmark Index

Author Index